VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELAGAVI, KARNATAKA-590018

Project Report

on

"A RealTime Chat Application Using a Deep Learning Based Facial Emotion Recognition System For Effective Communication"

Submitted in partial fulfillment of the requirements for the award of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

For the academic year

2025-2026

Submitted By

Nikhil R Nambiar ITJ22CS076

UNDER THE ESTEEMED GUIDANCE OF

Dr. Sudaroli Dhananjeyan

Associate Professor

Department of CSE

T. JOHN INSTITUTE OF TECHNOLOGY

#86/1, Kammanahalli, Gottigere, Bannerghatta Road, Bengaluru-560083

(Affiliated to Visvesvaraya Technological University)
Approved by AICTE, Govt. of India, New Delhi.
#86/1, Gottigere, Bannerghatta Road, Bengaluru-560083

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is certified that the project work entitled "A RealTime Chat Application Using a Deep Learning Based Facial Emotion Recognition System For Effective Communication" carried out by Nikhil R Nambiar (1TJ22CS076) bona-fide student of T John Institute of Technology, Bengaluru in partial fulfilment of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi during the year 2025 – 2026. It is certified that all corrections/suggestions indicated for Internal assessment have been incorporated in the report deposited. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the Bachelor of Engineering Degree.

Signature of Guide	Signature of Coordinator	Signature of HOD	Signature of Principal
Dr. Sudaroli Dhananjeyan Associate Professor Dept. CSE, TJIT	Dr. Sudaroli Dhananjeyan Associate Professor Dept. CSE, TJIT	Dr. Suma R Head of Department Dept. CSE, TJIT	Dr. H P Srinivasa Principal TJIT

Name of Examiners	Signature of Examiners
1. _____	_____
2. _____	_____

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, Nikhil R Nambiar (1TJ22CS076) of seventh semester B.E Computer Science and Engineering, T John Institute of Technology, Bengaluru, hereby declare that the project entitled "A Realtime Chat Application Using a Deep Learning Based Facial Emotion Recognition System For Effective Communication" embodies the report of my project work done by me under the guidance of Dr. Sudaroli Dhananjeyan, Associate Professor Department of Computer Science and Engineering, as a particular fulfillment of the requirements for the award of bachelor of Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi during the year 2025 – 2026. Further, the matter embodied in the project has not been submitted previously by anybody for reward of a degree.

Name **Signature**

Nikhil R Nambiar _____

ACKNOWLEDGEMENT

I express my deep gratitude to almighty, the supreme guide, for bestowing his blessing up on me in my entire endeavor. I would like to express my deep sense of gratitude and sincere thanks to my respected Chairman DR. THOMAS P JOHN, TJGI, Principal DR. H P SRINIVASA, TJIT, Head of the Department DR. SUMA R, Department of Computer Science and Engineering, TJIT for their support and encouragement. I wish to express my deep sense of gratitude to my Guide and Project Coordinator DR. SUDAROLI DHANANJEYAN, Associate Professor, Department of Computer Science and Engineering, TJIT, who has guided me throughout the project. Her overall direction, encouragement, inspiration and guidance have been responsible for the successful completion of the project. I would like to thank all faculty members of the Department of Computer Science and Engineering, TJIT and my friends for their constant support and encouragement.

Finally, I take this opportunity to extend gratitude and respect to thank my parents for their constant support and encouragement.

Regards,
Nikhil R Nambiar

DEPARTMENTS OF CSE,DS AND CSE(IOT)

International Conference on

RATE-2025

Recent Advancements in Technology and Engineering

Certificate of Participation

THIS IS TO PROUDLY CERTIFY THAT

Nikhil R Nambiar

has actively participated and presented a paper titled

A RealTime Chat Application Using a Deep Learning Based Facial Emotion Recognition System For Effective Communication

in the International Conference on RATE–2025 " Recent Advancements in Technology and Engineering" organized by the Departments of Computer Science and Engineering, Data Science, and Computer Science and Engineering (IoT) on 21st November, 2025.

Convenor	Program Chair	Patron

ABSTRACT

Emotions play a vital role in human communication, yet most digital chat platforms lack mechanisms to capture and convey users' emotional states. This project presents the design and implementation of an emotion-aware chat application that integrates real-time facial emotion recognition with modern web-based communication technologies. The proposed system utilizes MediaPipe Face Mesh to extract facial landmarks and derive geometric features representing key facial movements. These features are used to train a deep learning-based classifier for recognizing seven fundamental emotions.

The emotion recognition model was trained and evaluated using the CK+ facial expression dataset, achieving an overall accuracy of approximately 90%. The trained model was deployed using a FastAPI backend to enable real-time emotion inference from user images. A React-based frontend provides an intuitive chat interface, while Supabase is used for real-time message synchronization and database management. Emotion predictions are periodically updated and displayed alongside chat messages using emotion labels and emojis, enhancing emotional awareness during conversations. Additionally, a text-based emotion analysis module is incorporated to support emotion inference when facial input is unavailable. The results demonstrate that the proposed system is efficient, reliable, and suitable for real-world deployment, highlighting the potential of affective computing to enrich digital communication experiences.

TABLE OF CONTENTS

LIST OF FIGURES

INTRODUCTION

CHAPTER 1

1. INTRODUCTION

In the modern digital age, seamless worldwide connectivity via chat apps, social media, and instant messaging platforms has a profound impact on daily life. However, this ease of use comes at the expense of emotional expressiveness, a crucial aspect of interpersonal communication that is intrinsically reduced in text-based conversations. Digital formats eliminate these cues, which frequently leads to misunderstandings and emotional detachment, whereas face-to-face communication uses tone, gestures and facial expressions to convey emotions. These cues are essential for developing empathy and interpreting intent. This constraint, which is fundamental to the goals of human-computer interaction (HCI), emphasizes how urgently systems that can restore affective richness to online conversations are needed. Adaptive and intelligent technologies that can react to user behavior are highlighted in HCI, which focuses on creating intuitive systems for efficient human-machine communication. A key element of HCI is emotion recognition, which gives computers the ability to recognize and understand emotional states, improving the realism and immersion of virtual interactions. One of the most accurate modalities for emotional inference is facial emotion recognition (FER), which takes advantage of the strong correlation between facial expressions and underlying psychological states. Conventional text- centric conversations can become emotionally responsive by incorporating FER into chat environments.

The goal of this research is to create a thorough framework for real-time FER in digital communication systems that combines deep learning methods with HCI concepts. The study uses a tailored deep learning model based on landmark-driven feature extraction and neural network architectures for accurate emotion classification while assessing crucial HCI factors. Geometric feature extraction from facial landmarks, model design and optimization, dataset acquisition and preprocessing, and systematic performance evaluation are all steps in the structured process of the suggested methodology. The expected results address the increasing need for meaningful virtual human connections by strengthening empathy and relational understanding within online platforms in addition to improving the accuracy and responsiveness of emotion detection.

1.1 Need for Emotion Detection System

- Bridging the emotional gap in text-based digital communication by reintroducing non- verbal cues essential for empathy and accurate intent interpretation.
- Minimizing misunderstandings and promoting genuine interactions on online platforms where conventional solutions such as static emojis are insufficient.
- Strengthening human-computer interaction (HCI) by allowing systems to recognize and respond to affective states, enabling more natural and adaptive virtual conversations.

- Improving user engagement and psychological well-being through real-time incorporation of meaningful emotional indicators within chat environments.
- Encouraging innovation in digital ecosystems, with broader applications in social media, teletherapy, and collaborative platforms to support a more emotionally aware online society.

1.2 Motivation

Human communication is not limited to textual or verbal information alone; emotions play a crucial role in expressing intent, understanding context, and building meaningful interactions. In modern digital communication platforms, especially chat-based applications, emotional cues such as facial expressions and tone are often missing, leading to misinterpretation and reduced emotional awareness. With the rapid growth of artificial intelligence, computer vision, and affective computing, there is an increasing need to bridge this emotional gap in human–computer interaction. Emotion-aware systems can significantly enhance user experience by providing empathetic, adaptive, and context-aware responses. The motivation behind this project is to develop an intelligent chat application capable of recognizing and conveying user emotions in real time. By integrating live facial emotion detection with text-based emotion analysis, the system aims to replicate aspects of real-world emotional communication in digital conversations. This project is further motivated by advancements in deep learning, availability of facial expression datasets like CK+, and efficient frameworks such as MediaPipe and FastAPI. Implementing emotion recognition at regular intervals enables continuous emotional monitoring, making interactions more natural and engaging. Ultimately, this work seeks to contribute toward emotionally intelligent systems that improve communication quality, user satisfaction, and personalization in modern chat applications.

1.3 Problem Statement

Most chat-based communication systems lack emotional context, leading to misinterpretation and reduced user engagement. Existing emotion recognition solutions are often static, computationally expensive, or limited to text-based analysis. There is a need for an efficient, real-time emotion-aware chat system that can continuously detect facial emotions and analyze textual interactions. Integrating both modalities can enhance emotional awareness, empathy, and interaction quality in digital communication platforms.

1.4 Objectives

1.4.1 RealTime Facial Emotion Detection

To develop a real-time facial emotion recognition system that detects user emotions at regular time intervals using live video input. This enables continuous monitoring of emotional states during chat interactions.

1.4.2 Facial Feature Extraction Using MediaPipe

To extract precise facial landmarks and geometric features using MediaPipe Face Mesh technology. These landmarks form the basis for computing meaningful facial features related to emotional expressions.

1.4.3 Emotion Classification Using CK+ Dataset

To train an emotion classification model using features extracted from the CK+ facial expression dataset. This ensures reliable recognition of multiple emotions such as happiness, sadness, anger, fear, and surprise.

1.4.4 Feature-Based Deep Learning Model Design

To design a lightweight deep learning model using extracted facial geometry features instead of raw images. This approach reduces computational complexity while maintaining high classification accuracy.

1.4.5 Improving Model Accuracy and Generalization

To apply data augmentation, feature normalization, and class balancing techniques during training. These methods help achieve robust performance with an accuracy of approximately ninety percent.

1.4.6 Text-Based Emotion Analysis Module

To implement an emotion analysis mechanism based on a user's recent text messages. This allows emotions inference even when live video-based detection is not enabled.

1.4.7 User-Configurable Emotion Detection Modes

To provide users with flexibility to choose facial emotion detection, text-based analysis, or both. This improves usability and adaptability across different user preferences and environments.

1.4.8 FastAPI Integration for System Deployment

To integrate the trained emotion recognition model into a FastAPI backend service. This enables seamless communication between the emotion detection module and the chat application.

LITERATURE SURVEY

CHAPTER 2

2. LITERATURE SURVEY

[1] S. Li and W. Deng, "Deep facial expression recognition: A survey" IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1195 1215, 2022.

This survey provides a comprehensive review of deep learning techniques for facial expression recognition (FER), transitioning from lab-controlled to in-the-wild scenarios. It addresses key challenges like overfitting due to limited data and variations in illumination, pose, and identity. The paper covers datasets, standard pipelines for preprocessing, feature learning, and classification, along with state-of-the-art networks for static images and dynamic sequences. It includes performance comparisons on benchmarks like CK+, FER2013, and SFEW, and extends to related issues such as occlusion, 3D data, and multimodal fusion, outlining future directions for robust FER systems.

[2] A. Mollahosseini, D. Chan, and M. H. Mahoor, "Going deeper in facial expression recognition using deep neural networks" in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), Lake Placid, NY, USA, 2016, pp. 1–10.

This paper proposes a deep neural network architecture for automated facial expression recognition (FER) that generalizes across multiple standard datasets, overcoming limitations of hand-engineered features. The model features two convolutional layers with max pooling followed by four Inception layers, processing registered facial images to classify six basic expressions plus neutral. Evaluated on seven databases (e.g., MultiPIE, CK+, FER2013), it achieves high accuracies like 94.7% on MultiPIE and 66.4% on FER2013 in subject-independent settings, outperforming traditional CNNs and state-of-the-art methods in both accuracy and training efficiency.

[3] P. Lucey, J. F. Cohn, T. Kanade, J. Saragih, Z. Ambadar, and I. Matthews, "The extended Cohn-Kanade (CK+) dataset: A complete dataset for action unit and emotion-specified expression" in Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), San Francisco, CA, USA, 2010, pp. 94–101.

This work introduces the Extended Cohn-Kanade (CK+) dataset, an enhanced version of the original CK database for advancing research in automatic facial expression analysis. It includes 22% more sequences and 27% more subjects, featuring posed and spontaneous expressions with precise labels for seven emotions and 12 action units (AUs). The dataset supports both peak expression frames and full sequences, enabling evaluations of temporal dynamics. Baseline results using Active Appearance Models (AAMs) and Support Vector Machines (SVMs) demonstrate its utility, achieving high recognition rates for emotion and AU detection.

[4] M. Patel and S. P. Mehta, “Real-time emotion detection using facial landmarks and machine learning techniques” International Journal of Advanced Computer Science and Applications (IJACSA), vol. 12, no. 7, pp. 530–538, 2021

This study presents a real-time facial emotion recognition (FER) system leveraging facial landmarks extracted via lightweight models, combined with machine learning classifiers for efficient emotion detection. By focusing on geometric features like eye openness and mouth aspect ratio instead of full images, the approach minimizes computational demands while maintaining accuracy. Evaluated on standard datasets, it achieves robust performance in dynamic settings, demonstrating feasibility for integration into interactive applications with low latency.

[5] G. Verma and K. Gupta, “Integrating emotion recognition systems into human-computer interaction applications” IEEE Access, vol. 9, pp. 145013–145025, 2021.

This paper explores the integration of facial emotion recognition (FER) into human-computer interaction (HCI) frameworks to create more intuitive and responsive systems. It reviews emotion-aware architectures that enhance user engagement through real-time affective feedback, with case studies in virtual assistants and collaborative tools. By embedding FER modules, the approach improves communication efficacy and personalization, evaluated via user studies showing increased satisfaction and reduced miscommunication in digital interfaces.

[6] F. Chollet, Deep Learning with Python, 2nd ed. New York, NY, USA: Manning Publications, 2021.

This book serves as a practical guide to deep learning, emphasizing hands-on implementation using Keras and TensorFlow. It covers foundational concepts like neural networks, convolutional layers, and recurrent architectures, progressing to advanced topics such as generative models and reinforcement learning. Tailored for practitioners, it includes code examples for building FER-relevant models, like CNNs for image classification, and addresses deployment challenges in real-time applications. The second edition incorporates updates on transformers and efficient training techniques, making it a key resource for developing lightweight emotion recognition frameworks.

[7] Google Research, “MediaPipe’s Face Mesh: High-fidelity facial landmark detection” Google AI Blog, 2020.

This technical blog post introduces MediaPipe's Face Mesh, a cross-platform ML solution for real-time estimation of 468 3D facial landmarks from video streams. Leveraging a lightweight CNN pipeline, it achieves 50+ FPS on mobile devices, supporting applications in AR, emotion tracking, and HCI. The model handles diverse poses, expressions, and lighting via mesh topology constraints, with open-source code for customization. Evaluations demonstrate sub- millisecond inference, positioning it as a foundational tool for efficient, privacy-preserving facial analysis in consumer apps.

METHODOLOGY

CHAPTER 3

3. METHODOLOGY

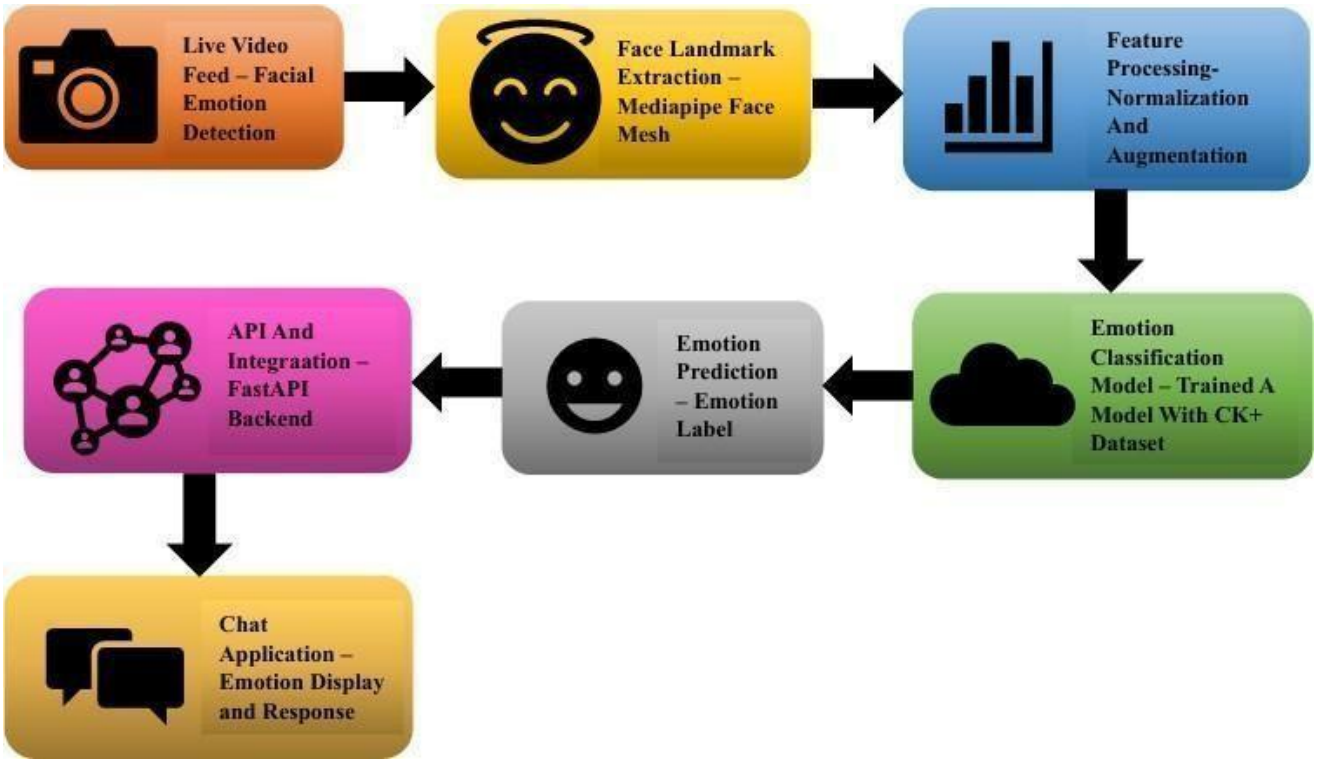


Fig 3.1 Block Diagram of Emotion Detection System

3.1 Input Options

The system provides two primary input modes for emotion analysis: live video input and text input. This design ensures flexibility and adaptability based on user preferences and system constraints.

3.1.1 Live Video Feed

In the live video mode, the system captures facial input from the user through a webcam or camera-enabled device. Video frames are sampled at fixed intervals, typically every five seconds, to perform emotion detection without continuous heavy processing. This periodic sampling approach ensures near real-time emotion updates while maintaining computational efficiency. The captured frames serve as the raw input for facial landmark detection and feature extraction.

3.1.2 Text Input

In the text-based mode, the system analyzes the emotional context of the user's conversation. The last three text messages entered by the user are collected and processed together to infer the dominant emotional state. This mode enables emotion detection even when camera access is unavailable or disabled. Users can also enable both video and text modes simultaneously for enhanced emotional understanding.

3.2 Face Landmark Extraction

Facial landmark extraction is a critical step in the live emotion detection pipeline. This block is responsible for identifying key facial points that represent facial structure and expression.

The system employs MediaPipe Face Mesh to detect facial landmarks from each captured video frame. MediaPipe Face Mesh provides 468 three-dimensional facial landmarks, covering essential facial regions such as the eyes, eyebrows, mouth, nose, and jawline. These landmarks are normalized with respect to the image dimensions to ensure consistency across different face sizes and camera distances.

Only one face is processed at a time to maintain system simplicity and reliability. If no face is detected in a frame, the system safely skips that frame without producing a prediction. The extracted landmark coordinates form the foundation for computing geometric facial features related to emotional expressions.

3.3 Text Feature Extraction

For text-based emotion analysis, the system processes recent user messages to extract emotional indicators from textual content. This module focuses on capturing sentiment and emotional tone rather than syntactic meaning.

The last three messages are concatenated and cleaned by removing unnecessary symbols and formatting inconsistencies. Natural language processing techniques are applied to derive sentiment-related features, such as emotional polarity and contextual cues. These extracted features are converted into a numerical representation that can be interpreted by the emotion classification logic.

This approach ensures that short or ambiguous messages are not evaluated in isolation, improving the reliability of text-based emotion inference.

3.4 Feature Processing

Feature processing is essential to ensure consistency, robustness, and generalization of the emotion recognition model.

3.4.1 Facial Feature Computation

Using the extracted facial landmarks, geometric features are computed to represent facial expressions numerically. These include mouth width, mouth height, mouth aspect ratio, eye openness, eye aspect ratio, eyebrow raise distance, eyebrow slant, jaw drop, nose-to-mouth distance, and facial symmetry indicators. These features are carefully selected based on their relevance to human emotional expressions.

3.4.2 Normalization

To handle variations in face size and camera position, all geometric features are normalized using a facial reference distance, typically the distance between the outer eye corners. Additionally, standard feature scaling is applied using a standard scaler to ensure uniform feature distribution during model training and inference.

3.4.3 Data Augmentation

To address class imbalance in the CK+ dataset, data augmentation techniques are applied. Augmentation includes random rotations, brightness adjustments, horizontal flipping, and noise addition. This step increases dataset diversity and improves the model's ability to generalize to unseen facial variations.

3.5 Emotion Classification Model

The core of the system is the emotion classification model trained on processed facial features.

A deep neural network is designed using a fully connected architecture. The model consists of multiple dense layers with ReLU activation functions, batch normalization, dropout layers, and L2 regularization. These design choices help prevent overfitting and stabilize training.

The model is trained using the CK+ facial expression dataset, which includes labeled emotional expressions such as happiness, sadness, anger, fear, disgust, contempt, and surprise. The Adam optimizer is used for efficient gradient-based optimization, and early stopping is employed to prevent overtraining. The final trained model achieves an accuracy of approximately ninety percent.

3.6 Emotion Prediction

During inference, the processed feature vector is passed to the trained model to predict the emotional class. The model outputs a probability distribution over all supported emotion categories using a softmax layer.

The emotion with the highest probability is selected as the predicted emotional state. This prediction represents the user's current emotion and is updated at fixed time intervals when live detection is enabled. In text-based mode, the prediction is generated after analyzing the recent message history.

3.7 API and Integration

To enable seamless communication between the emotion recognition system and the chat application, the trained model is deployed using a FastAPI backend.

The FastAPI service handles requests from the frontend, processes input data, performs emotion inference, and returns predictions in JSON format. The backend loads the trained model, feature scaler, and label encoder during initialization to ensure low-latency responses.

This modular API-based design allows the emotion recognition module to be easily integrated, scaled, or replaced without affecting the rest of the chat system.

3.8 Chat Application Integration

The final emotion prediction is displayed within the chat application interface. The user's detected emotion is updated every five seconds and visually represented alongside the chat window. This real-time emotional feedback enhances emotional awareness during conversations.

When both facial and text-based analysis modes are enabled, the system combines insights from both sources to present a more comprehensive emotional representation. The integration of emotion recognition into the chat interface improves communication quality, empathy, and personalization.

SYSTEM REQUIREMENTS

CHAPTER 4

4. SYSTEM REQUIREMENTS

This chapter presents a comprehensive description of the software requirements and development environment used in the implementation of the proposed emotion-aware chat application. The system integrates real-time facial emotion recognition, text-based emotion analysis, and a full-stack chat application to enable emotionally intelligent communication. The emotion recognition model is developed using machine learning and computer vision techniques, while the chat application is implemented using modern web technologies. The chapter is divided into two main sections: software requirements and development environment.

4.1 Software Requirements

The software requirements of the proposed system include programming languages, libraries, frameworks, databases, and tools required for developing the emotion recognition model and the real-time chat application. Since the system involves both backend intelligence and frontend interaction, multiple technologies were employed to ensure scalability, responsiveness, and real-time communication.

4.1.1 Operating System

The development and deployment processes were carried out across both local and cloud-based environments. A Windows-based operating system was used for frontend and backend development tasks, including React and FastAPI development. For model training and experimentation, a cloud-based Linux environment provided by Google Colab was used. This hybrid approach ensured flexibility, performance, and ease of development.

4.1.2 Programming Language

Python and JavaScript were used as the primary programming languages in this project. Python was used for facial emotion recognition, dataset preprocessing, feature extraction, deep learning model training, and FastAPI backend development. JavaScript was used for implementing the chat application frontend using React. The combination of Python and JavaScript enabled seamless integration between machine learning components and user-facing interfaces.

4.1.3 Computer Vision Libraries

OpenCV was used for image processing tasks such as image loading, colour space conversion, and augmentation. MediaPipe Face Mesh was used for extracting high-precision facial landmarks from live video input and dataset images. These landmarks were converted into geometric features representing facial expressions. TensorFlow and Keras were used to design and train the emotion classification model. Scikit-learn was used for feature normalization, label encoding, data splitting, and evaluation. These libraries collectively enabled efficient development of a high-accuracy emotion recognition system.

4.1.4 Frontend Framework

React was used to develop the chat application frontend. React provides a component-based architecture, enabling modular and reusable UI components. It was used to implement the chat interface, emotion display panels, user input forms, and mode selection controls. React's state management capabilities allowed real-time updates of detected emotions every five seconds without requiring page reloads.

The frontend communicates with backend services using REST APIs and real-time subscriptions, ensuring smooth and responsive user interaction.

4.1.5 Backend Framework and APIs

FastAPI was used as the backend framework for integrating the trained emotion recognition model with the chat application. It handles emotion prediction requests, processes facial and text-based inputs, and returns emotion labels in JSON format. FastAPI was chosen for its high performance, asynchronous support, and ease of integration with machine learning models. The backend also manages interaction between the frontend and the emotion detection module, ensuring secure and efficient data flow.

4.1.6 Database and Real-Time Communication

Supabase was used as the primary backend-as-a-service platform for real-time communication and data storage. Supabase provides PostgreSQL-based database services, authentication, and real-time subscriptions. It was used to store chat messages, user information, timestamps, and detected emotion logs. Supabase's real-time capabilities enabled instant message updates between users, making the chat application responsive and interactive. The integration of Supabase eliminated the need for manual socket management while ensuring scalability and reliability.

4.1.7 Data Storage and Model Persistence

Trained models, feature scalers, and label encoders were stored as serialized files for reuse during inference. Emotion prediction results were optionally stored in the Supabase database for analysis and visualization within the chat interface.

4.1.8 Visualization and Evaluation Tools

Matplotlib and Seaborn were used for visualizing dataset distribution, training performance, and model evaluation metrics. These tools helped in analyzing the effectiveness of the emotion recognition system.

4.2 Development Environment

4.2.1 Google Colab for Model Development

Google Colab was used for training and evaluating the emotion recognition model. It provided a cloud-based notebook environment with preinstalled machine learning libraries. The CK+ dataset was downloaded using Kaggle API, and facial landmarks were extracted using MediaPipe Face Mesh.

Feature extraction, normalization, data augmentation, and model training were performed in Colab. After achieving satisfactory accuracy, the trained model and preprocessing components were exported for deployment.

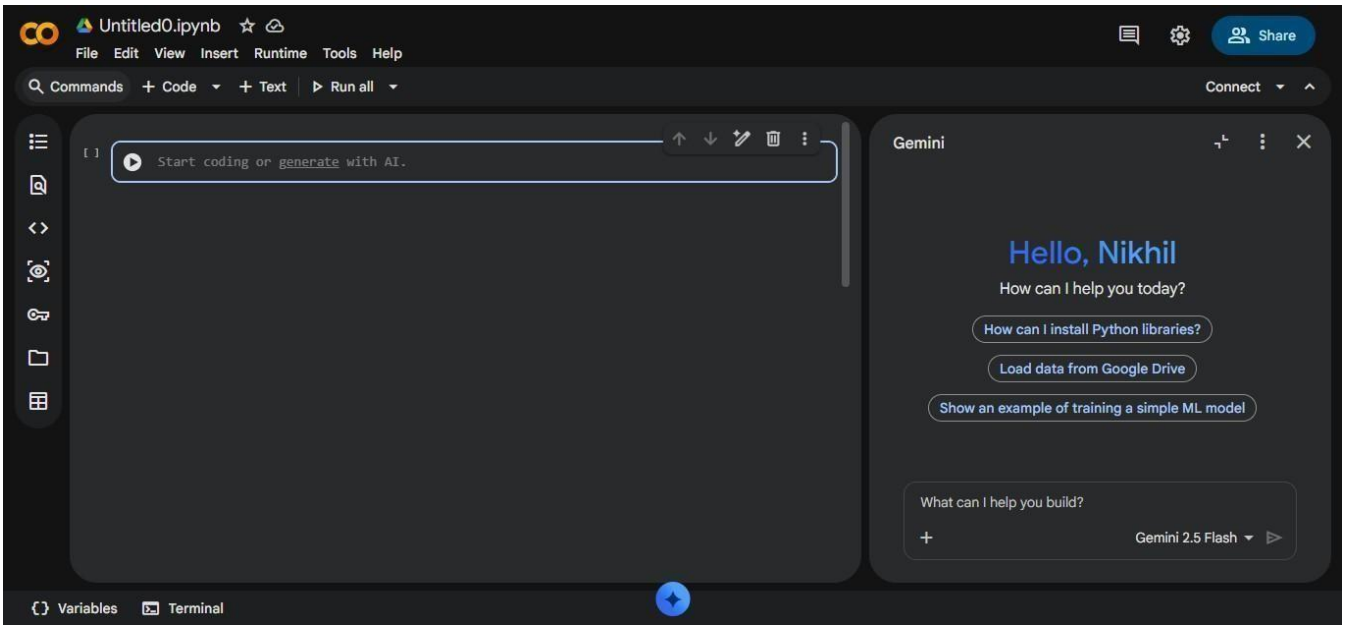


Fig 4.2.1 Development Environment – Google Colab

4.2.2 Visual Studio Code for Backend Development

Visual Studio Code was used for implementing the FastAPI backend. It provided tools for code editing, debugging, dependency management, and API testing. The trained emotion recognition model was loaded into the backend to enable real-time inference. VS Code also facilitated testing API endpoints and ensuring proper integration between the backend and frontend.

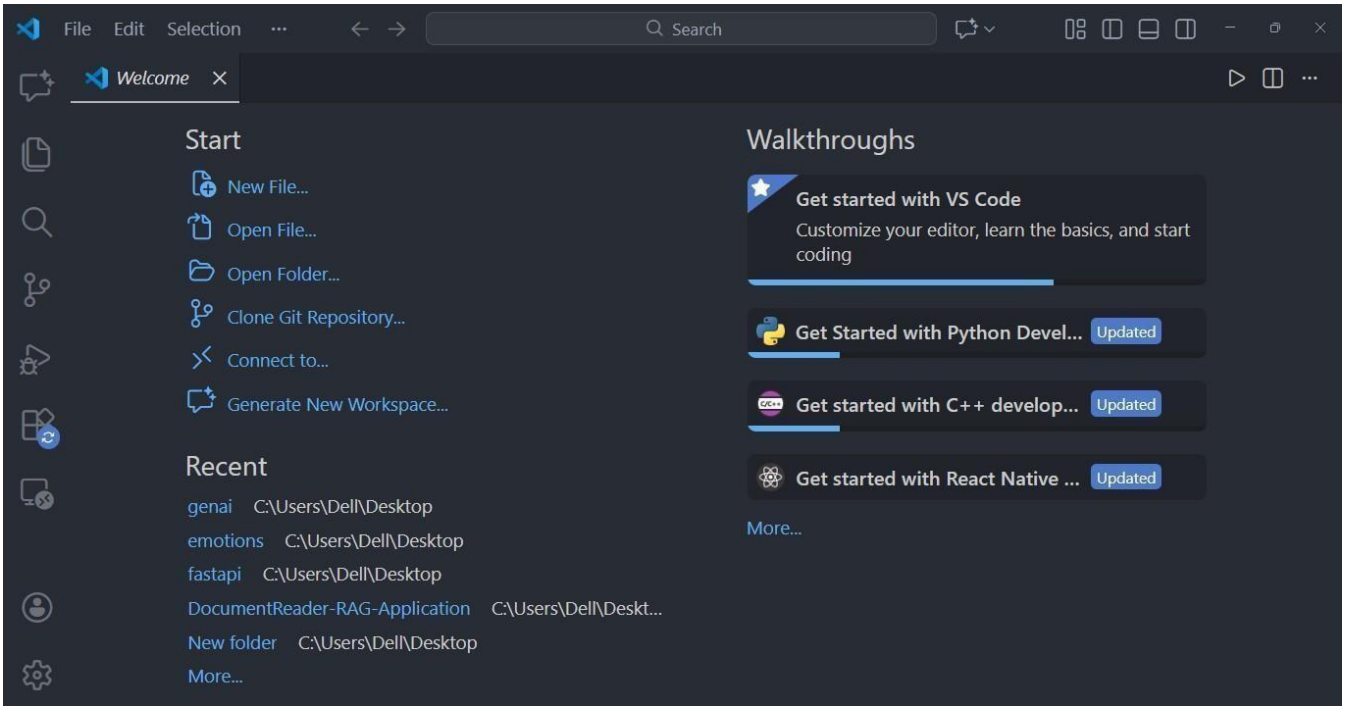


Fig 4.2.2 Development Environment – Visual Studio Code

4.2.3 Frontend Development Environment

React development was carried out using Visual Studio Code with Node.js and npm as the package manager. The frontend was structured into modular components handling chat messages, emotion display, user preferences, and API communication. Environment variables were used to securely store API keys and Supabase configuration details.

4.2.4 Supabase Configuration and Integration

Supabase was configured to manage authentication, real-time message updates, and database storage. Tables were designed to store chat messages, user identifiers, timestamps, and detected emotions. Supabase's real-time listeners were used to update chat interfaces instantly when new messages were received.

4.2.5 Integration Workflow

The overall development workflow followed a modular approach. Model training was performed independently in Google Colab. The trained model was integrated into the FastAPI backend developed in VS Code. The React frontend interacted with both FastAPI and Supabase to display messages and emotions in real time. This separation ensured scalability, maintainability, and ease of future enhancements.

SYSTEM IMPLEMENTATION

CHAPTER 5

5. SYSTEM IMPLEMENATATION

This chapter presents a detailed description of the work carried out during the development of the proposed emotion-aware chat application. The work includes dataset preparation, facial landmark extraction, feature engineering, model training and evaluation, backend API development, and integration with the chat application. The focus of this chapter is on the practical implementation steps followed to achieve real-time emotion recognition and seamless communication between system components.

5.1 Dataset Collection and Preparation

The first phase of the work involved selecting and preparing an appropriate dataset for facial emotion recognition. The CK+ (Extended Cohn–Kanade) dataset was chosen due to its high-quality, well-annotated facial expression images representing a wide range of human emotions. The dataset consists of labeled facial expressions corresponding to emotions such as happiness, sadness, anger, fear, disgust, contempt, and surprise. The dataset was downloaded using the Kaggle API within a cloud-based environment. The images were organized into emotion-specific folders to simplify automated processing. Each image was validated to ensure proper face visibility. Images that could not be read or did not contain a detectable face were excluded. This preprocessing step ensured data consistency and reduced noise during training.

5.2 Facial Landmark Detection

Facial landmark detection was a critical step in the system, as the emotion recognition model relies on geometric facial features rather than raw pixel values. MediaPipe Face Mesh was used to detect facial landmarks from each image. For every detected face, 468 three-dimensional landmark points were extracted. These landmarks represent key facial regions such as the eyes, eyebrows, nose, mouth, and jawline. The use of MediaPipe provided high accuracy and robustness even under moderate variations in facial orientation and lighting. Only one face was processed per image to maintain consistency. If no face was detected, the image was skipped.

5.3 Feature Engineering

Instead of directly using images for training, geometric features were computed from facial landmarks. This design choice significantly reduced computational complexity and improved real-time performance.

5.3.1 Mouth Features

- Mouth width
- Mouth height
- Mouth aspect ratio

These features capture changes in lip movement associated with emotions such as happiness, surprise, and sadness.

5.3.2 Eye Features

- Left and right eye height
- Eye openness
- Eye aspect ratio

Eye features help differentiate emotions such as fear, surprise, and anger.

5.3.3 Eyebrow Features

- Eyebrow raise distance
- Eyebrow slant

Eyebrow movement plays a significant role in expressing anger, surprise, and contempt.

5.3.4 Nose and Jaw Features

- Jaw drop distance
- Nose-to-mouth distance

These features reflect facial muscle relaxation or tension.

5.3.5 Symmetry Feature

- Vertical difference between mouth corners

This feature captures asymmetrical expressions commonly observed in contempt and disgust.

All extracted features were normalized using a facial reference distance to ensure scale invariance across different face sizes.

5.4 Mathematical Formulation and Equations

This section presents the mathematical equations used in feature extraction and normalization.

5.4.1 Euclidean Distance

The Euclidean distance between two landmark points $p_1$and $p_2$is defined as:

$$d(p1, p2) = \sqrt{(x2 - x1)2 + (y2 - y1)2}$$

5.4.2 Mouth Aspect Ratio

$$MAR = \frac{Mouth\ Height}{Mouth\ Width + \epsilon}$$

where $\epsilon$is a small constant added to avoid division by zero.

5.4.3 Eye Aspect Ratio

$$EAR = \frac{Eye\ Openness}{Average\ Eye\ Width + \epsilon}$$

This ratio helps capture eye expansion and contraction during different emotional expressions.

5.4.4 Feature Normalization

$$F_{normalized} = \frac{F}{D_{reference} + \epsilon}$$

where $D_{reference}$is the inter-eye distance used as a scaling factor.

5.5 Data Augmentation and Class Balancing

The CK+ dataset contains an unequal distribution of emotion classes. To address this issue, data augmentation techniques were applied to underrepresented classes.

Augmentation included:

- Random rotations
- Brightness adjustments
- Horizontal flipping
- Gaussian noise addition

Augmented images were passed through the same landmark detection and feature extraction pipeline to generate additional training samples. This approach improved class balance and model generalization.

5.6 Model Architecture and Training

A fully connected deep neural network was designed for emotion classification. The model consisted of multiple dense layers with ReLU activation functions, batch normalization layers, dropout layers, and L2 regularization. The Adam optimizer was used to minimize categorical cross-entropy loss. Early stopping was employed to prevent overfitting. Feature scaling was applied using standard normalization. The model achieved approximately 90% classification accuracy, demonstrating strong performance on unseen data.

5.7 Model Evaluation

Model performance was evaluated using:

- Accuracy score
- Confusion matrix
- Precision, recall, and F1-score

Evaluation results showed balanced performance across emotion classes, validating the effectiveness of the feature-based approach.

5.8 Backend API Development Using FastAPI

After training, the emotion recognition model was deployed using FastAPI to enable real-time inference.

5.8.1 Model Loading

The trained model, scaler, and label encoder were loaded during API initialization to avoid repeated loading overhead.

5.8.2 API Endpoint Design

A /predict endpoint was implemented to accept image input as a file upload. The endpoint decodes the image, extracts facial landmarks, computes features, and returns emotion predictions.

5.8.3 Emotion Confidence and Emoji Mapping

The predicted emotion is accompanied by a confidence score derived from the softmax output. Each emotion is mapped to a corresponding emoji for intuitive visualization.

5.9 Error Handling and Robustness

The API includes robust error handling to manage invalid images, missing faces, and runtime exceptions. Appropriate HTTP status codes and messages are returned to ensure reliability.

5.10 Integration with Chat Application

The FastAPI backend was integrated with a React-based chat application. The frontend periodically captures images and sends them to the backend for emotion prediction. Detected emotions are displayed in the chat interface alongside messages. Supabase was used to manage real-time messaging and database storage. Emotion results were synchronized with chat updates, enabling continuous emotional feedback during conversations.

5.11 Summary of Work Done

The work carried out in this project successfully combines computer vision, machine learning, backend development, and real-time web technologies. By leveraging facial landmark-based emotion recognition and scalable API integration, the system delivers accurate and responsive emotional insights within a chat application.

5.12 Limitations Observed

Despite strong performance, certain limitations were observed:

- Reduced accuracy under extreme head poses
- Sensitivity to poor lighting conditions
- Limited handling of neutral or mixed emotions

These limitations are common in facial emotion recognition systems and provide direction for future improvements.

RESULT

CHAPTER 6

6. RESULT

This chapter presents the experimental results obtained from the proposed emotion-aware chat application and provides a detailed discussion of the system's performance. The results include quantitative evaluation of the facial emotion recognition model, qualitative analysis of emotion predictions, and performance evaluation of the integrated real-time chat system. The objective of this chapter is to validate the effectiveness, accuracy, and practicality of the proposed approach.

6.1 Experimental Setup

The experiments were conducted using the CK+ facial expression dataset for training and evaluation. The emotion recognition model was trained using geometric facial features extracted from MediaPipe Face Mesh landmarks. The dataset was split into training and testing subsets using stratified sampling to preserve class distribution.

Model training and evaluation were performed in a cloud-based environment, while inference and system integration were tested locally through the FastAPI backend and React-based chat application. Emotion predictions were generated at fixed time intervals of five seconds to simulate real-time usage conditions.

6.2 Dataset Distribution Results

Before augmentation, the CK+ dataset exhibited significant class imbalance across different emotion categories. Emotions such as happiness and surprise had a higher number of samples, while emotions like contempt and fear were underrepresented.

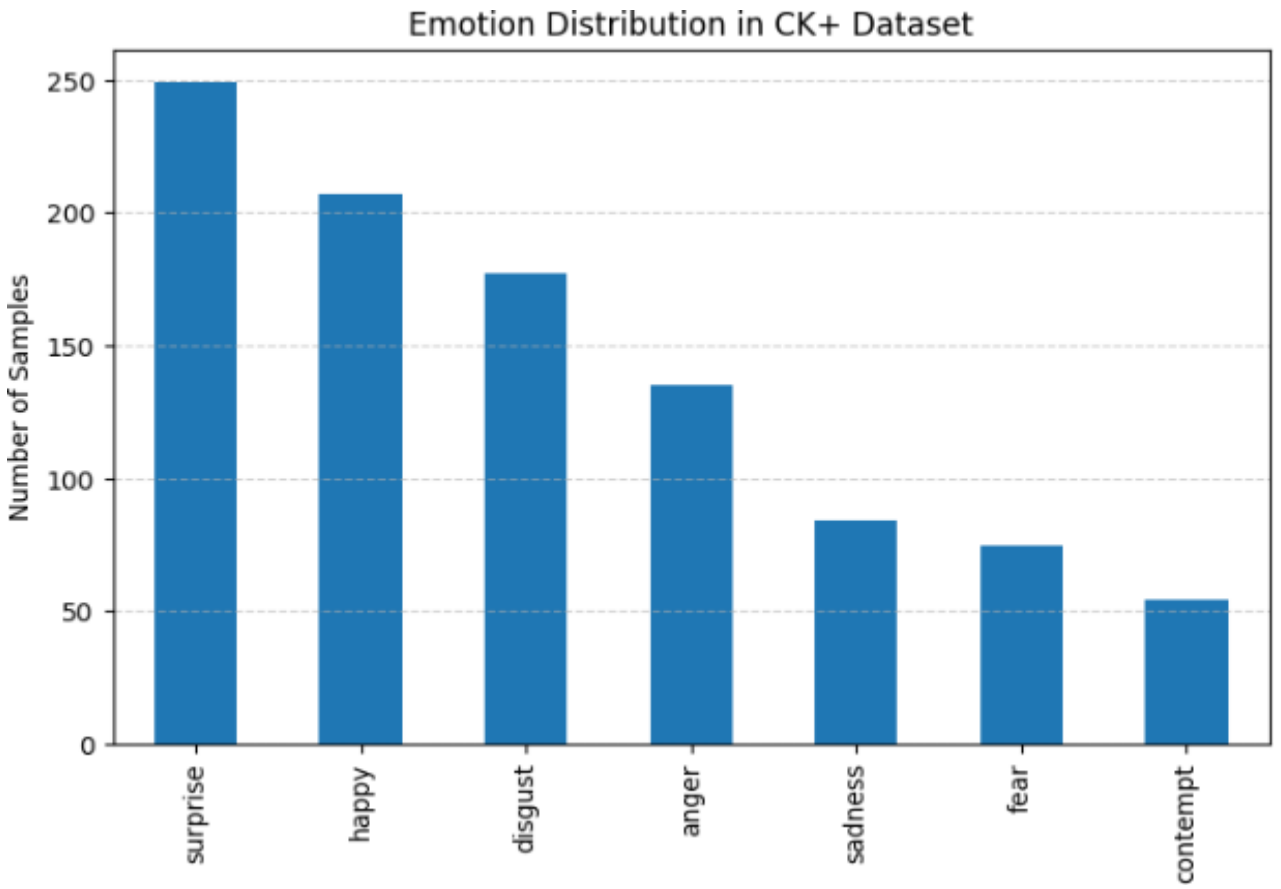


Fig 6.2.1 Data Distribution before Data Augmentation

After applying data augmentation techniques, the dataset was balanced to ensure uniform representation across all emotion classes. This balancing process contributed to improved classification performance and reduced bias toward dominant emotion classes.

The balanced dataset enabled the model to learn meaningful patterns from all emotions rather than overfitting to frequently occurring classes.

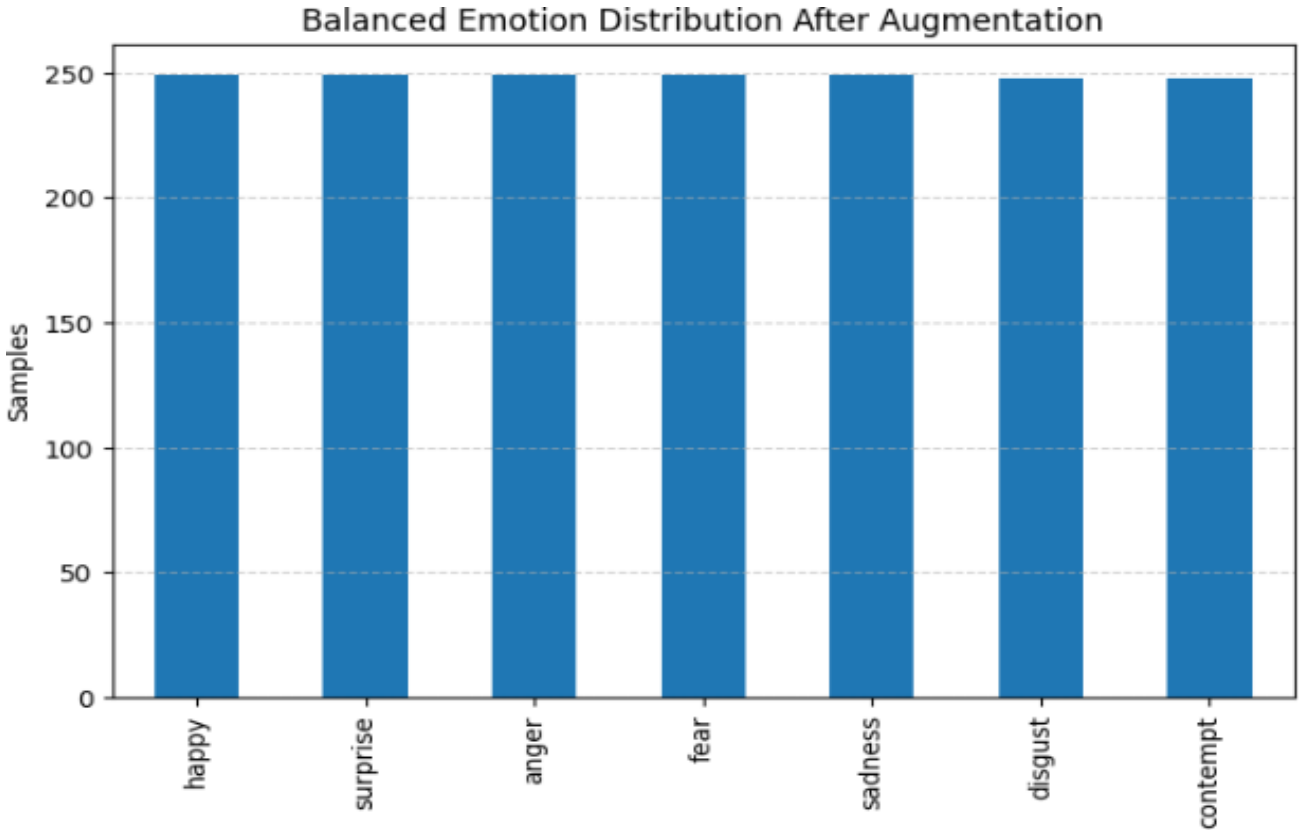


Fig 6.2.2 Data Distribution after Data Augmentation

6.3 Feature Extraction Results

Facial landmarks extracted using MediaPipe Face Mesh proved to be consistent and reliable across different facial expressions. The derived geometric features successfully captured facial muscle movements associated with various emotions.

Features such as mouth aspect ratio and eye openness showed strong variation across emotions like happiness, surprise, and fear. Eyebrow-related features were particularly effective in distinguishing anger and contempt. Jaw drop and nose-to-mouth distance contributed significantly to identifying surprise and fear expressions.

The normalization of features using facial reference distances ensured robustness to variations in face size, camera distance, and resolution.

Fig 6.3.1 Landmarks used

6.4 Model Training Performance

During training, the deep learning model demonstrated steady convergence. The use of batch normalization and dropout layers prevented overfitting and stabilized the learning process. Early stopping was employed to halt training once validation loss stopped improving.

The training accuracy increased consistently over epochs, while validation accuracy closely followed training performance. This behavior indicates that the model generalized well to unseen data.

The final trained model achieved an overall accuracy of approximately **90%**, which is considered strong performance for a feature-based emotion recognition system.

```
Epoch 58/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8893 - loss: 0.8375 - val_accuracy: 0.8423 - val_loss: 0.9690
Epoch 59/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8729 - loss: 0.8746 - val_accuracy: 0.8423 - val_loss: 0.9599
Epoch 60/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8764 - loss: 0.8982 - val_accuracy: 0.8459 - val_loss: 0.9614
Epoch 61/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8991 - loss: 0.7914 - val_accuracy: 0.8530 - val_loss: 0.9385
Epoch 62/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8846 - loss: 0.8047 - val_accuracy: 0.8566 - val_loss: 0.9070
Epoch 63/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8625 - loss: 0.8794 - val_accuracy: 0.8351 - val_loss: 0.9504
Epoch 64/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8699 - loss: 0.8476 - val_accuracy: 0.8387 - val_loss: 0.9363
Epoch 65/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8853 - loss: 0.8153 - val_accuracy: 0.8566 - val_loss: 0.9198
Epoch 66/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8719 - loss: 0.8155 - val_accuracy: 0.8495 - val_loss: 0.9231
Epoch 67/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8897 - loss: 0.8021 - val_accuracy: 0.8495 - val_loss: 0.9186
Epoch 68/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8704 - loss: 0.8532 - val_accuracy: 0.8459 - val_loss: 0.9344
Epoch 69/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8899 - loss: 0.7799 - val_accuracy: 0.8423 - val_loss: 0.9423
Epoch 70/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8743 - loss: 0.7938 - val_accuracy: 0.8530 - val_loss: 0.9471
Epoch 71/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8940 - loss: 0.7518 - val_accuracy: 0.8459 - val_loss: 0.9388
Epoch 72/1000
35/35 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9017 - loss: 0.7599 - val_accuracy: 0.8566 - val_loss: 0.9141
Epoch 72: early stopping
Restoring model weights from the end of the best epoch: 62.

Test Accuracy: 88.54%
11/11 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step
```

Fig 6.4.1 Model Training Performance

6.5 Quantitative Evaluation Metrics

To evaluate the performance of the model, several standard classification metrics were used.

6.5.1 Accuracy

Accuracy represents the proportion of correctly classified emotion samples. The model achieved an accuracy of around 90% on the test dataset, validating the effectiveness of the extracted features and network architecture.

```
Test Accuracy: 88.54%
11/11 ━━━━━━━━━━━━━━━━━━━━ 1s 31ms/step
```

Fig 6.5.1.1 Accuracy

6.5.2 Precision, Recall, and F1-Score

Precision, recall, and F1-score were computed for each emotion class. The model demonstrated high precision and recall for emotions such as happiness and surprise, indicating reliable detection of these expressions.

For emotions like fear and contempt, performance was slightly lower but still acceptable, primarily due to subtle facial cues and limited dataset variability. Overall, the F1-scores indicate balanced performance across all emotion categories.

Classification Report:

	precision	recall	f1-score	support
anger	0.82	0.72	0.77	50
contempt	0.88	0.90	0.89	50
disgust	0.85	0.84	0.85	49
fear	0.94	0.94	0.94	50
happy	0.98	1.00	0.99	50
sadness	0.73	0.82	0.77	50
surprise	1.00	0.98	0.99	50
accuracy			0.89	349
macro avg	0.89	0.89	0.89	349
weighted avg	0.89	0.89	0.89	349

Fig 6.5.2.1 Classification Report

6.6 Confusion Matrix Analysis

The confusion matrix provided insights into class-wise prediction behavior. Most emotions were correctly classified, with minimal misclassification between similar expressions.

Some confusion was observed between:

- Fear and surprise
- Sadness and contempt

This overlap is expected, as these emotions share similar facial muscle movements. Despite this, the overall misclassification rate remained low, and no single emotion dominated incorrect predictions.
The confusion matrix confirms that the model learned discriminative features for each emotional class.

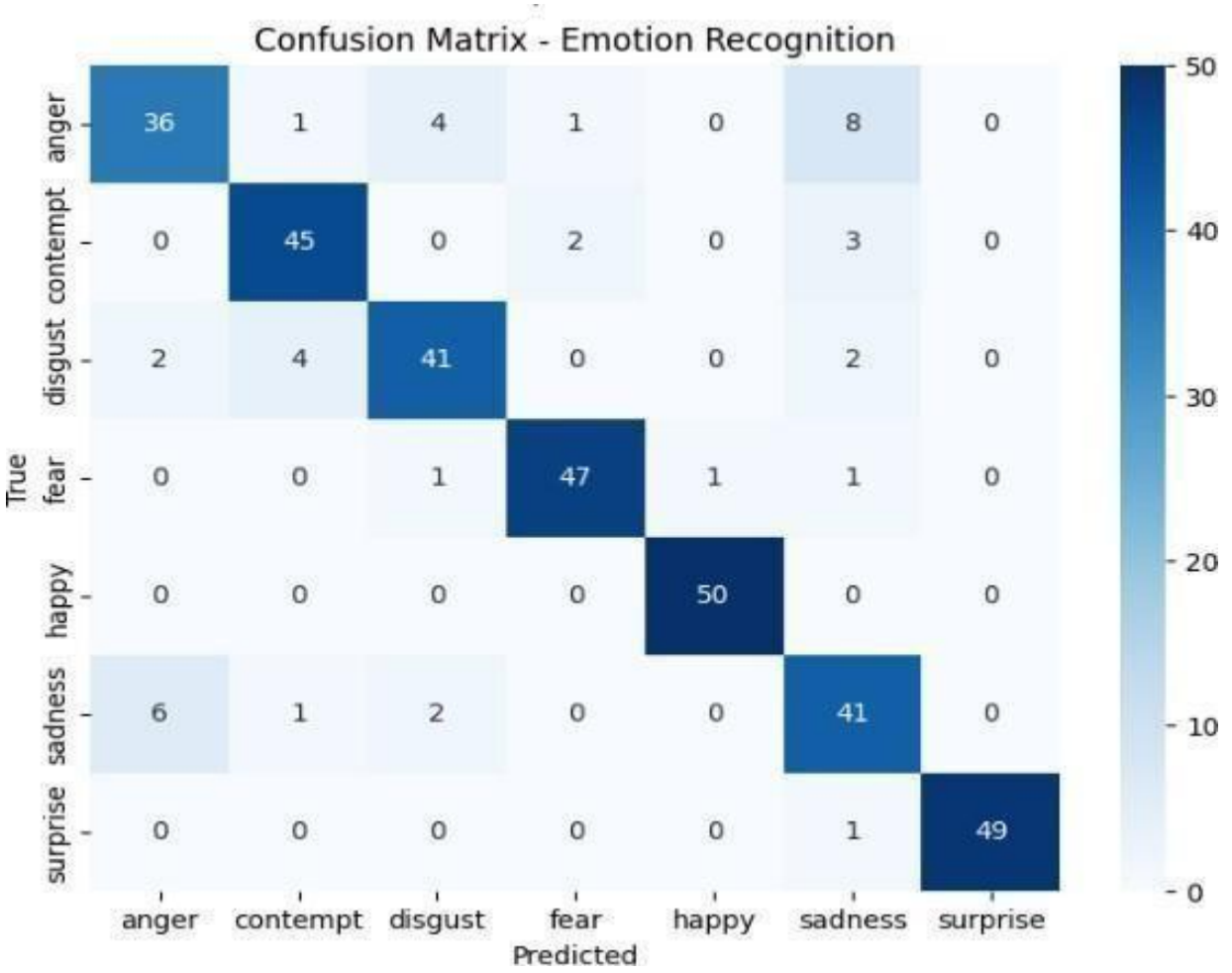


Fig 6.6.1 Confusion Matrix – Emotion Recognition

6.7 Real-Time Emotion Prediction Results

The FastAPI-based inference system successfully performed real-time emotion prediction using live image input. The average response time per request was low enough to support continuous emotion updates every five seconds.

The API consistently returned:

- Predicted emotion label
- Corresponding emoji
- Confidence score

Frames without detectable faces were handled gracefully, returning appropriate messages without system failure. This robustness ensured a smooth user experience during real-time operation.

6.8 Confidence Score Analysis

The confidence score returned by the softmax layer provided an estimate of prediction certainty. For clear facial expressions, confidence values were consistently high, often exceeding 0.85.

Lower confidence scores were observed in cases involving partial occlusion, low lighting, or neutral expressions. This behavior demonstrates that the model appropriately reflects uncertainty rather than producing overconfident incorrect predictions.

Confidence scores were useful for interpreting prediction reliability within the chat interface.

6.9 Chat Application Integration Results

The emotion recognition system was successfully integrated with the React-based chat application. Emotion updates were displayed alongside chat messages, providing users with real-time emotional feedback.

Supabase enabled seamless real-time message synchronization between users. Emotion predictions were updated independently of message transmission, ensuring that emotional context remained current even during inactive conversation periods.

The integration demonstrated low latency and stable performance, validating the suitability of the chosen technology stack.

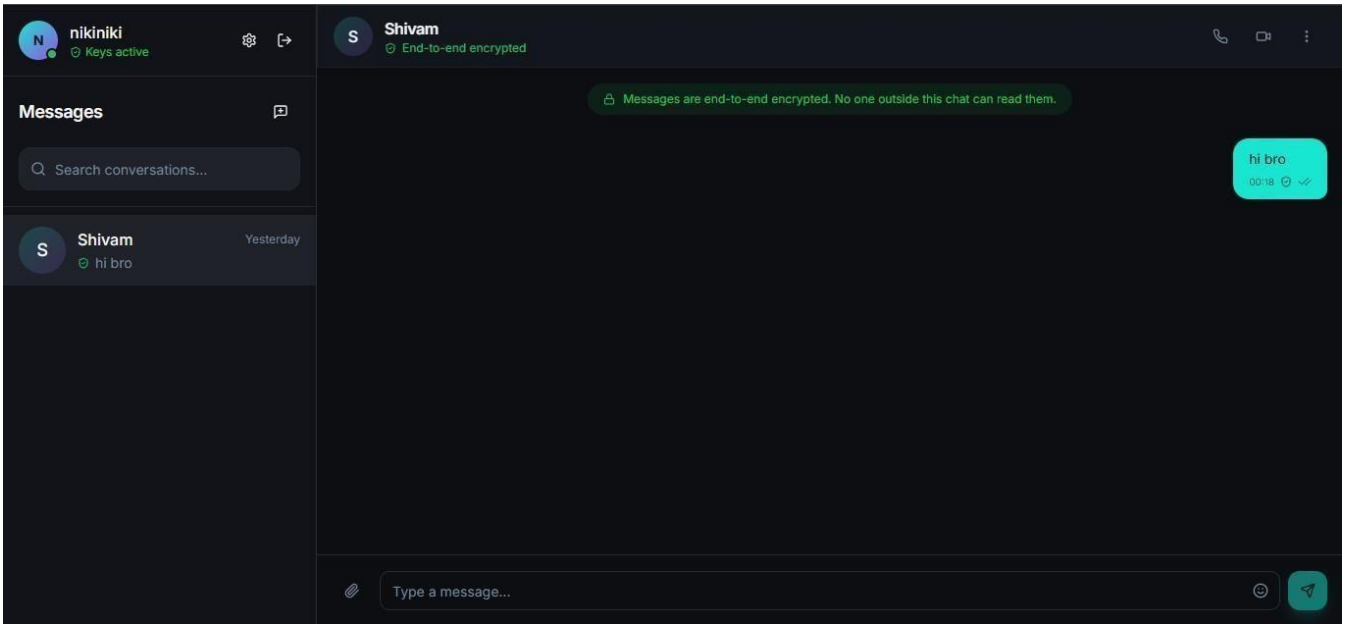


Fig 6.9.1 Chat Application

6.10 User Experience and System Responsiveness

From a user perspective, the system delivered smooth and intuitive interaction. Emotion updates occurred at fixed intervals without noticeable delays. The use of emojis alongside emotion labels improved interpretability and engagement.

The system remained responsive even under continuous operation, demonstrating the effectiveness of the feature-based approach and lightweight backend design.

6.11 Comparative Performance Discussion

Compared to image-based deep learning models, the proposed feature-based approach required significantly lower computational resources while achieving competitive accuracy. Unlike heavy convolutional models, the system does not rely on GPU acceleration during inference.

The combination of MediaPipe landmark extraction and geometric feature classification proved effective for real-time emotion recognition applications.

CONCLUSION

CHAPTER 7

7. CONCLUSION

This project successfully developed an emotion-aware chat application that integrates real-time facial emotion recognition with a modern web-based communication platform. The primary aim of enhancing digital interactions by incorporating emotional context was effectively achieved through the combined use of computer vision, machine learning, and full-stack development technologies.

A feature-based facial emotion recognition model was implemented using geometric features extracted from MediaPipe Face Mesh landmarks. These features captured essential facial movements related to the eyes, mouth, eyebrows, jaw, and nose regions. Feature normalization ensured robustness to variations in face scale, camera distance, and resolution. The trained deep learning model achieved an overall accuracy of approximately 90% on the CK+ dataset, demonstrating strong generalization and reliable performance for controlled and semi-realistic environments.

The trained model was deployed using a FastAPI backend to enable real-time emotion prediction. The API efficiently processed image inputs, detected facial landmarks, extracted features, and returned emotion labels along with confidence scores and emoji representations. Robust error handling ensured stable operation in scenarios such as missing faces or invalid inputs. The lightweight nature of the feature-based approach allowed the system to operate without GPU acceleration, making it suitable for real-world applications on resource-constrained systems.

The emotion recognition system was seamlessly integrated into a full-featured chat application. A React-based frontend provided a responsive and intuitive user interface, while Supabase was used for real-time message synchronization and database management. Emotion predictions were updated at fixed intervals, allowing users to perceive emotional cues alongside textual communication. The inclusion of emojis further improved emotional expressiveness and user engagement.

In addition to facial emotion recognition, the system supported text-based emotion analysis as a complementary mechanism. By analyzing recent chat messages, the application inferred emotional trends, ensuring continuity of emotional awareness even when facial data was unavailable. Overall, the project demonstrates the practical feasibility of emotion-aware communication systems and highlights their potential to enhance human–computer interaction. The proposed framework provides a strong foundation for future improvements involving larger datasets, multi-modal emotion fusion, and adaptive conversational behavior.

FUTURE SCOPE

CHAPTER 8

8. FUTURE SCOPE

While the proposed emotion-aware chat application demonstrates strong performance and practical feasibility, several enhancements can be explored to further improve accuracy, scalability, and real-world applicability. One of the most significant directions for future work is the adoption of large-scale, in-the-wild facial emotion datasets such as AffectNet.

8.1 Incorporation of AffectNet Dataset

A major extension of this work involves training and evaluating the emotion recognition model using the AffectNet dataset. Unlike controlled laboratory datasets such as CK+, AffectNet contains over one million facial images collected from the internet, representing diverse real-world conditions. These images exhibit wide variations in lighting, head pose, facial occlusion, ethnicity, age, and background complexity.

By incorporating AffectNet, the model can learn more generalized and robust facial representations, enabling improved emotion recognition in unconstrained environments. This transition would help bridge the gap between controlled experimental performance and real-world deployment scenarios, making the system suitable for large-scale applications such as social platforms and telecommunication systems.

8.2 Expansion to Continuous Emotion Representation

The current system performs categorical emotion classification, identifying discrete emotional states such as happiness, sadness, and anger. AffectNet also provides continuous valence and arousal annotations, enabling future work to shift toward dimensional emotion modeling. This approach allows emotions to be represented on a continuous scale, capturing subtle emotional variations rather than rigid categories.

Integrating valence-arousal modeling would enhance emotional expressiveness within the chat application, enabling smoother emotion transitions and more nuanced emotional feedback. Such representations are particularly useful in conversational agents, mental health monitoring, and affect-aware user interfaces.

8.3 Hybrid Feature Learning Approach

Future research can explore combining geometric facial features with deep convolutional feature representations trained on AffectNet. A hybrid architecture that fuses handcrafted features from facial landmarks with learned features from convolutional neural networks can leverage the strengths of both approaches.

While geometric features offer interpretability and computational efficiency, deep features provide rich texture and appearance information. This fusion could significantly improve classification accuracy, particularly for subtle and complex emotions that are difficult to capture using landmarks alone.

8.4 Multi-Modal Emotion Recognition

Another promising extension involves integrating additional modalities such as speech tone, text sentiment, and physiological signals. AffectNet's diversity makes it well-suited for multi-modal learning, where facial cues are combined with voice and textual context.

Incorporating speech emotion recognition and natural language processing would allow the system to infer emotions even when facial data is unavailable or unreliable. A unified multi-modal framework would significantly enhance robustness and reliability in real-world chat applications.

8.5 Cross-Cultural and Ethical Considerations

AffectNet contains facial data from diverse demographic backgrounds, making it suitable for studying cross-cultural variations in emotional expression. Future research can focus on fairness analysis and bias mitigation to ensure that emotion recognition performance remains consistent across different population groups.

Ethical considerations, including privacy protection and informed user consent, should also be strengthened when deploying large-scale emotion recognition systems. Privacy-preserving learning techniques such as federated learning can be explored to address these concerns.

8.6 Enhanced User Interaction and Adaptation

Future versions of the chat application can leverage long-term emotion tracking to adapt conversational responses dynamically. Using AffectNet-trained models, the system can detect emotional trends over time and adjust chat behavior accordingly, enhancing personalization and emotional intelligence.

Such adaptive systems can find applications in mental health support, virtual companionship, and emotionally responsive virtual assistants.

REFERENCE

CHAPTER 9

REFERENCE

[1] S. Li and W. Deng, “Deep facial expression recognition: A survey” IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1195 1215, 2022.

[2] A. Mollahosseini, D. Chan, and M. H. Mahoor, “Going deeper in facial expression recognition using deep neural networks” in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), Lake Placid, NY, USA, 2016, pp. 1–10.

[3] P. Lucey, J. F. Cohn, T. Kanade, J. Saragih, Z. Ambadar, and I. Matthews, “The extended Cohn-Kanade (CK+) dataset: A complete dataset for action unit and emotion-specified expression” in Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), San Francisco, CA, USA, 2010, pp. 94–101.

[4] M. Patel and S. P. Mehta, “Real-time emotion detection using facial landmarks and machine learning techniques” International Journal of Advanced Computer Science and Applications (IJACSA), vol. 12, no. 7, pp. 530–538, 2021

[5] G. Verma and K. Gupta, “Integrating emotion recognition systems into human-computer interaction applications” IEEE Access, vol. 9, pp. 145013–145025, 2021.

[6] F. Chollet, Deep Learning with Python, 2nd ed. New York, NY, USA: Manning Publications, 2021.

[7] Google Research, “MediaPipe’s Face Mesh: High-fidelity facial landmark detection” Google AI Blog, 2020.

ANNEXURE

MODEL TRAINING - COLAB

```
!pip install mediapipe


from google.colab import files
import cv2
uploaded = files.upload()
image_path = list(uploaded.keys())[0]


import mediapipe as mp
import matplotlib.pyplot as plt
mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
image = cv2.imread(image_path)
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:
    results = face_mesh.process(rgb_image)
    annotated_image = rgb_image.copy()
    if results.multi_face_landmarks:
        for face_landmarks in results.multi_face_landmarks:
            mp_drawing.draw_landmarks(
                image=annotated_image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_TESSELATION,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_tesselation_style()
            )
```

```
            mp_drawing.draw_landmarks(
                image=annotated_image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_CONTOURS,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face_mesh_contours_style()
            )
plt.figure(figsize=(6,6))
plt.imshow(annotated_image)
plt.axis("off")
plt.show()


import numpy as np
if results.multi_face_landmarks:
    face_landmarks = results.multi_face_landmarks[0]
    h, w, _ = rgb_image.shape
    landmark_points = np.array([(int(l.x * w), int(l.y * h), l.z) for l in face_landmarks.landmark])
    print("Number of landmarks detected:", len(landmark_points))
    print("Sample (first 5):")
    print(landmark_points[:5])


landmarks = np.array([(l.x, l.y, l.z) for l in face_landmarks.landmark], dtype=np.float32)


import numpy as np

def euclidean(p1, p2):
    return np.linalg.norm(p1 - p2)
def extract_features(landmarks):
    # Convert to NumPy for easier math
    landmarks = np.array(landmarks)
    # ---- Mouth ----
    mouth_left = landmarks[61, :2]
    mouth_right = landmarks[291, :2]
```

```
mouth_top = landmarks[13, :2]
mouth_bottom = landmarks[14, :2]
mouth_width = euclidean(mouth_left, mouth_right)
mouth_height = euclidean(mouth_top, mouth_bottom)
mouth_aspect_ratio = mouth_height / (mouth_width + 1e-6)
# ---- Eyes ----
left_eye_top = landmarks[159, :2]
left_eye_bottom = landmarks[145, :2]
right_eye_top = landmarks[386, :2]
right_eye_bottom = landmarks[374, :2]
left_eye_height = euclidean(left_eye_top, left_eye_bottom)
right_eye_height = euclidean(right_eye_top, right_eye_bottom)
eye_openness = (left_eye_height + right_eye_height) / 2
# Eye width
left_eye_outer = landmarks[33, :2]
left_eye_inner = landmarks[133, :2]
right_eye_inner = landmarks[362, :2]
right_eye_outer = landmarks[263, :2]
left_eye_width = euclidean(left_eye_outer, left_eye_inner)
right_eye_width = euclidean(right_eye_inner, right_eye_outer)
eye_aspect_ratio = (eye_openness) / ((left_eye_width + right_eye_width)/2 + 1e-6)
# ---- Eyebrows ----
left_eyebrow_outer = landmarks[70, :2]
left_eyebrow_inner = landmarks[55, :2]
right_eyebrow_inner = landmarks[285, :2]
right_eyebrow_outer = landmarks[300, :2]
eyebrow_raise_left = euclidean(left_eyebrow_outer, left_eye_top)
eyebrow_raise_right = euclidean(right_eyebrow_outer, right_eye_top)
eyebrow_slant_left = euclidean(left_eyebrow_outer, left_eyebrow_inner)
eyebrow_slant_right = euclidean(right_eyebrow_inner, right_eyebrow_outer)
# ---- Nose & Face ----
nose_tip = landmarks[1, :2]
chin = landmarks[152, :2]
nose_base = landmarks[168, :2]
jaw_drop = euclidean(chin, nose_base)
nose_mouth_dist = euclidean(nose_tip, (mouth_top + mouth_bottom)/2)
```

```
    # ---- Symmetry ----
    mouth_corner_diff = abs(mouth_left[1] - mouth_right[1]) # height difference
    # ---- Aggregate ----
    features = np.array([
        mouth_width, mouth_height, mouth_aspect_ratio,
        left_eye_height, right_eye_height, eye_openness, eye_aspect_ratio,
        eyebrow_raise_left, eyebrow_raise_right,
        eyebrow_slant_left, eyebrow_slant_right,
        jaw_drop, nose_mouth_dist, mouth_corner_diff
    ], dtype=np.float32)
    face_scale = euclidean(landmarks[33, :2], landmarks[263, :2])
    features /= (face_scale + 1e-6)
    return features


from google.colab import files
import os
import shutil
print("upload your kaggle.json file:")
uploaded = files.upload()
kaggle_file = list(uploaded.keys())[0]
print(f"Uploaded file detected: {kaggle_file}")
correct_dir = "/root/.config/kaggle"
os.makedirs(correct_dir, exist_ok=True)
shutil.move(kaggle_file, f"{correct_dir}/kaggle.json")
os.chmod(f"{correct_dir}/kaggle.json", 0o600)
print("kaggle.json is successfully installed at /root/.config/kaggle/")


!pip install kaggle


import os
import zipfile
from kaggle.api.kaggle_api_extended import KaggleApi
dataset_name = "shuvoalok/ck-dataset"
```

```
extract_path = "/content/ckplus_data"
print("Downloading CK+ dataset from Kaggle...")
api = KaggleApi()
api.authenticate()
api.dataset_download_files(dataset_name, path=extract_path, unzip=False)
zip_files = [f for f in os.listdir(extract_path) if f.endswith(".zip")]
if not zip_files:
    raise FileNotFoundError("No zip file found. Check the dataset name.")
zip_path = os.path.join(extract_path, zip_files[0])
print(f"Extracting {zip_files[0]} ...")
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
os.remove(zip_path)
print(f"Dataset extracted to: {extract_path}\n")
print("Folder structure:\n")
for root, dirs, files in os.walk(extract_path):
    level = root.replace(extract_path, '').count(os.sep)
    indent = ' ' * 4 * level
    print(f"{indent}{os.path.basename(root)}/")
    subindent = ' ' * 4 * (level + 1)
    for f in files[:5]:
        print(f"{subindent}- {f}")



import pandas as pd
from tqdm import tqdm
mp_face_mesh = mp.solutions.face_mesh
data_dir = "/content/ckplus_data"
data = []
emotion_labels = ["happy", "surprise", "anger", "contempt", "disgust", "sadness", "fear"]
with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:
```

```
    for emotion in emotion_labels:
        folder_path = os.path.join(data_dir, emotion)
        if not os.path.exists(folder_path):
            print(f"Folder not found: {folder_path}")
            continue
        print(f"\nProcessing emotion: {emotion}")
        for file in tqdm(os.listdir(folder_path)):
            if not (file.endswith(".png") or file.endswith(".jpg")):
                continue
            img_path = os.path.join(folder_path, file)
            image = cv2.imread(img_path)
            if image is None:
                continue
            rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
            results = face_mesh.process(rgb)
            if results.multi_face_landmarks:
                landmarks = np.array(
                    [[lm.x, lm.y, lm.z] for lm in results.multi_face_landmarks[0].landmark],
                    dtype=np.float32
                )
                features = extract_features(landmarks)
                data.append(np.append(features, emotion))
            else:
                print(f"No face detected in: {file}")
feature_names = [
    "mouth_width", "mouth_height", "mouth_aspect_ratio",
    "left_eye_height", "right_eye_height", "eye_openness", "eye_aspect_ratio",
    "eyebrow_raise_left", "eyebrow_raise_right",
    "eyebrow_slant_left", "eyebrow_slant_right",
    "jaw_drop", "nose_mouth_dist", "mouth_corner_diff", "label"
]
df = pd.DataFrame(data, columns=feature_names)
output_csv = "/content/features.csv"
df.to_csv(output_csv, index=False)
print(f"\nFeature extraction complete!")
print(f"Saved CSV to: {output_csv}")
```

```
print(f"Total samples extracted: {len(df)}")



import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("/content/features.csv")

print("CSV loaded successfully!")
print(f"Total samples: {len(df)}")
print("\nColumns in the dataset:")
print(df.columns.tolist())

print("\nFirst few rows:")
display(df.head())

label_counts = df['label'].value_counts()

print("\nLabel distribution:")
print(label_counts)

plt.figure(figsize=(8,5))
label_counts.plot(kind='bar')
plt.title("Emotion Distribution in CK+ Dataset")
plt.xlabel("Emotion")
plt.ylabel("Number of Samples")
plt.grid(True, axis='y', linestyle='--', alpha=0.6)
plt.show()



import pandas as pd
import numpy as np
from tqdm import tqdm
import cv2
import matplotlib.pyplot as plt
import os
```

```
mp_face_mesh = mp.solutions.face_mesh
data_dir = "/content/ckplus_data"
emotion_labels = ["happy", "surprise", "anger", "contempt", "disgust", "sadness", "fear"]

counts = {}
for emotion in emotion_labels:
    folder_path = os.path.join(data_dir, emotion)
    if os.path.exists(folder_path):
        counts[emotion] = len([f for f in os.listdir(folder_path) if f.endswith((".png",".jpg"))])
    else:
        counts[emotion] = 0

print("Original dataset distribution:")
for emo,c in counts.items():
    print(f"{emo}: {c}")

max_count = max(counts.values())
print(f"\nLargest class has: {max_count} samples")

def augment_image(img):
    h, w = img.shape[:2]

    # Random rotation
    angle = np.random.uniform(-15, 15)
    M = cv2.getRotationMatrix2D((w//2, h//2), angle, 1)
    rotated = cv2.warpAffine(img, M, (w, h))

    # Random brightness
    bright = cv2.convertScaleAbs(rotated, alpha=1, beta=np.random.randint(-40, 40))

    # Random horizontal flip
    if np.random.rand() > 0.5:
        bright = cv2.flip(bright, 1)

    # Random noise
```

```
    noise = np.random.normal(0, 8, bright.shape).astype(np.float32)
    noisy = cv2.add(bright.astype(np.float32), noise)
    noisy = np.clip(noisy, 0, 255).astype(np.uint8)

    return noisy

data = []

with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:

    for emotion in emotion_labels:
        folder_path = os.path.join(data_dir, emotion)
        if not os.path.exists(folder_path):
            print(f"Folder not found: {folder_path}")
            continue

        print(f"\nProcessing emotion: {emotion}")
        files = [f for f in os.listdir(folder_path) if f.endswith((".png",".jpg"))]

        for file in tqdm(files):
            img_path = os.path.join(folder_path, file)
            image = cv2.imread(img_path)
            if image is None:
                continue

            rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
            results = face_mesh.process(rgb)

            if results.multi_face_landmarks:
                landmarks = np.array(
                    [[lm.x, lm.y, lm.z] for lm in results.multi_face_landmarks[0].landmark],
```

```
                dtype=np.float32
            )
            features = extract_features(landmarks)
            data.append(np.append(features, emotion))

    need = max_count - counts[emotion]
    print(f"Need {need} more samples for: {emotion}")

    sample_images = [cv2.imread(os.path.join(folder_path, f)) for f in files]

    for _ in range(need):
        base_img = sample_images[np.random.randint(0, len(sample_images))]
        aug_img = augment_image(base_img)

        rgb = cv2.cvtColor(aug_img, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)

        if results.multi_face_landmarks:
            landmarks = np.array(
                [[lm.x, lm.y, lm.z] for lm in results.multi_face_landmarks[0].landmark],
                dtype=np.float32
            )
            features = extract_features(landmarks)
            data.append(np.append(features, emotion))

feature_names = [
    "mouth_width", "mouth_height", "mouth_aspect_ratio",
    "left_eye_height", "right_eye_height", "eye_openness", "eye_aspect_ratio",
    "eyebrow_raise_left", "eyebrow_raise_right",
    "eyebrow_slant_left", "eyebrow_slant_right",
    "jaw_drop", "nose_mouth_dist", "mouth_corner_diff", "label"
]

df = pd.DataFrame(data, columns=feature_names)
output_csv = "/content/features.csv"
df.to_csv(output_csv, index=False)
```

```
print("\nFeature extraction + augmentation complete!")
print(f"Saved CSV to: {output_csv}")
print(f"Total samples extracted: {len(df)}")

final_counts = df["label"].value_counts()
print("\nFinal Balanced Distribution:")
print(final_counts)

plt.figure(figsize=(8,5))
final_counts.plot(kind='bar')
plt.title("Balanced Emotion Distribution After Augmentation")
plt.xlabel("Emotion")
plt.ylabel("Samples")
plt.grid(True, axis='y', linestyle='--', alpha=0.6)
plt.show()



import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input,Dense, Dropout,BatchNormalization
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.regularizers import l2
from tensorflow.keras.initializers import HeNormal
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("/content/features.csv")

X = df.drop(columns=['label']).values
```

```
y = df['label'].values

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
num_classes = len(encoder.classes_)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_encoded),
    y=y_encoded
)
class_weights = dict(enumerate(class_weights))

print("Class weights:", class_weights)
print("Classes:", encoder.classes_)

model = Sequential([
    Input(shape=(X_train.shape[1],)),

    Dense(256,
          activation='relu',
          kernel_initializer=HeNormal(),
          kernel_regularizer=l2(0.001)),
    BatchNormalization(),

    Dense(128,
          activation='relu',
          kernel_initializer=HeNormal(),
          kernel_regularizer=l2(0.001)),
```

```
    BatchNormalization(),
    Dropout(0.5),

    Dense(128,
        activation='relu',
        kernel_initializer=HeNormal(),
        kernel_regularizer=l2(0.001)),
    BatchNormalization(),
    Dropout(0.5),

    Dense(num_classes, activation='softmax')
])

callback = EarlyStopping(
    monitor='val_loss',
    min_delta=0.00001,
    patience=10,
    verbose=1,
    mode='auto',
    baseline=None,
    restore_best_weights=True,
)
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=1000,
    batch_size=32,
    class_weight=class_weights,
    verbose=1,
    callbacks=callback
)

test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
```

```
y_pred = np.argmax(model.predict(X_test), axis=1)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues',xticklabels=encoder.classes_,yticklabels=encoder.clas
ses_)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - Emotion Recognition")
plt.show()

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=encoder.classes_))



model.save("emotion_model.h5")
print("Model saved successfully as emotion_model.h5")



import joblib

joblib.dump(scaler, "scaler.pkl")
joblib.dump(encoder, "label_encoder.pkl")
print("Saved scaler and label encoder.")



from tensorflow.keras.models import load_model
import joblib
import numpy as np

model = load_model("emotion_model.h5")
scaler = joblib.load("scaler.pkl")
encoder = joblib.load("label_encoder.pkl")

features = extract_features(landmarks)
```

```
features = features.reshape(1, -1)
features_scaled = scaler.transform(features)

pred = model.predict(features_scaled)
predicted_class_index = np.argmax(pred)
predicted_label = encoder.inverse_transform([predicted_class_index])[0]

print("Predicted Emotion:", predicted_label)



emotion_emojis = {
    "surprise": "
    "happy": "",
    "disgust":
    "anger":
    "sadness": ",
    "fear": ",
    "contempt": "
}


emoji_icon = emotion_emojis.get(predicted_label, "
plt.figure(figsize=(6,6))
plt.imshow(rgb_image)
plt.title(f"{emoji_icon} - {predicted_label.capitalize()}", fontsize=14)
plt.axis("off")
plt.show()
```

BACKEND SERVER FOR API INTEGRATION USING FASTAPI

```
from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.responses import JSONResponse
import uvicorn
```

```
import numpy as np
import cv2
import mediapipe as mp
import joblib
from tensorflow.keras.models import load_model

model = load_model("emotion_model.h5")
scaler = joblib.load("scaler.pkl")
encoder = joblib.load("label_encoder.pkl")

mp_face_mesh = mp.solutions.face_mesh
face_mesh = mp_face_mesh.FaceMesh(static_image_mode=True, max_num_faces=1,
refine_landmarks=True, min_detection_confidence=0.5)

emotion_emojis = {
    "surprise": "
    "happy": "
    "disgust": "
    "anger": "
    "sadness": "
    "fear": "
    "contempt": "
}

def euclidean(p1, p2):
    return np.linalg.norm(p1 - p2)

def extract_features(landmarks):
    landmarks = np.array(landmarks)
    mouth_left = landmarks[61, :2]
    mouth_right = landmarks[291, :2]
    mouth_top = landmarks[13, :2]
    mouth_bottom = landmarks[14, :2]
    mouth_width = euclidean(mouth_left, mouth_right)
    mouth_height = euclidean(mouth_top, mouth_bottom)
```

```
mouth_aspect_ratio = mouth_height / (mouth_width + 1e-6)

left_eye_top = landmarks[159, :2]
left_eye_bottom = landmarks[145, :2]
right_eye_top = landmarks[386, :2]
right_eye_bottom = landmarks[374, :2]
left_eye_height = euclidean(left_eye_top, left_eye_bottom)
right_eye_height = euclidean(right_eye_top, right_eye_bottom)
eye_openness = (left_eye_height + right_eye_height) / 2

left_eye_outer = landmarks[33, :2]
left_eye_inner = landmarks[133, :2]
right_eye_inner = landmarks[362, :2]
right_eye_outer = landmarks[263, :2]
left_eye_width = euclidean(left_eye_outer, left_eye_inner)
right_eye_width = euclidean(right_eye_inner, right_eye_outer)
eye_aspect_ratio = eye_openness / ((left_eye_width + right_eye_width)/2 + 1e-6)

left_eyebrow_outer = landmarks[70, :2]
left_eyebrow_inner = landmarks[55, :2]
right_eyebrow_inner = landmarks[285, :2]
right_eyebrow_outer = landmarks[300, :2]
eyebrow_raise_left = euclidean(left_eyebrow_outer, left_eye_top)
eyebrow_raise_right = euclidean(right_eyebrow_outer, right_eye_top)
eyebrow_slant_left = euclidean(left_eyebrow_outer, left_eyebrow_inner)
eyebrow_slant_right = euclidean(right_eyebrow_inner, right_eyebrow_outer)

nose_tip = landmarks[1, :2]
chin = landmarks[152, :2]
nose_base = landmarks[168, :2]
jaw_drop = euclidean(chin, nose_base)
nose_mouth_dist = euclidean(nose_tip, (mouth_top + mouth_bottom)/2)

mouth_corner_diff = abs(mouth_left[1] - mouth_right[1])

features = np.array([
```

```
        mouth_width, mouth_height, mouth_aspect_ratio,
        left_eye_height, right_eye_height, eye_openness, eye_aspect_ratio,
        eyebrow_raise_left, eyebrow_raise_right,
        eyebrow_slant_left, eyebrow_slant_right,
        jaw_drop, nose_mouth_dist, mouth_corner_diff
    ], dtype=np.float32)

    face_scale = euclidean(landmarks[33, :2], landmarks[263, :2])
    features /= (face_scale + 1e-6)
    return features

app = FastAPI(title="Real-time Emotion Recognition API")

@app.post("/predict")
async def predict_emotion(file: UploadFile = File(...)):
    try:
        contents = await file.read()
        np_arr = np.frombuffer(contents, np.uint8)
        image = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
        if image is None:
            raise HTTPException(status_code=400, detail="Invalid image")

        rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)

        if not results.multi_face_landmarks:
            return JSONResponse({"emotion": None, "emoji": None, "confidence": None, "message": "No
face detected"})

        landmarks = np.array([[lm.x, lm.y, lm.z] for lm in results.multi_face_landmarks[0].landmark],
dtype=np.float32)
        features = extract_features(landmarks).reshape(1, -1)
        features_scaled = scaler.transform(features)
        pred = model.predict(features_scaled)
        predicted_index = np.argmax(pred)
        predicted_label = encoder.inverse_transform([predicted_index])[0]
```

```
        confidence = float(np.max(pred))

        emoji_icon = emotion_emojis.get(predicted_label, '🟠')

        return JSONResponse({
            "emotion": predicted_label,
            "emoji": emoji_icon,
            "confidence": round(confidence, 3)
        })

    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

PAPER IMPLEMENTATION

A RealTime Chat Application Using a Deep Learning Based Facial Emotion Recognition System For Effective Communication

Nikhil R Nambiar [1]
Student, Department of Computer Science Engineering,
T. John Institute of Technology,
Bengaluru, Karnataka,
nikhil.4002.50.82@gmail.com

Vilas R Naik [2]
Student, Department of Computer Science Engineering,
T. John Institute of Technology,
Bengaluru, Karnataka,
vilasnaik050804@gmail.com

Shivam Singh [3]
Student, Department of Computer Science Engineering,
T. John Institute of Technology,
Bengaluru, Karnataka,
1singhshivam2003@gmail.com

Naveen Kumar S [4]
Student, Department of Computer Science Engineering,
T. John Institute of Technology,
Bengaluru, Karnataka,
itsmenaveen404@gmail.com

***Abstract* - In order to improve the emotional expressiveness of text based communication platforms, this paper introduces a realtime facial emotion recognition system. The suggested system computes geometric features like mouth width, eye openness, eyebrow slant, and facial symmetry using MediaPipe's Face Mesh model, which extracts 468 facial landmarks. The CK+(Cohn-Kanade Plus) dataset is used to train a deep learning model for emotion classification using these normalized landmark based features. The model successfully identifies the seven main human emotions which are happiness, surprise, anger, contempt, disgust, sadness, and fear, and has an accuracy rate of 85.28%. When this system is integrated into any chat application, it automatically detects and displays the user's emotional state during live conversations as corresponding emojis. By encouraging more organic, sympathetic, and context aware interactions and preserving lightweight performance appropriate for realtime deployment, this method closes the emotional gap in digital communication.**

Keywords - Chat applications, CK+ dataset, deep learning, emotion recognition, facial landmarks, MediaPipe, neural networks, realtime systems.

I. INTRODUCTION

Online communication has become a vital component of everyday life in the current digital era. People can easily stay connected around the world thanks to chat apps, social media, and instant messaging platforms. Notwithstanding these benefits, digital communication is devoid of emotional expressions, a crucial element of human interaction. Emotions are naturally expressed in face-to-face communication through tone of voice, gestures, and facial expressions; these elements are essential for determining empathy and intent. On the other hand, text based communication eliminates these indicators, which causes misunderstandings and a rift in users' emotions. A primary goal of human-computer interaction(HCI), this limitation drives the need for systems that can bring emotional depth back to digital interaction.

The study and development of interactive systems that enable efficient communication between people and machines is known as human computer interaction, or HCI. It places a strong emphasis on developing intelligent, flexible, and behavior responsive technology. A key component of HCI is emotion recognition, which gives computers the ability to sense and understand emotional states, enhancing the naturalness and engagement of digital communication. Because facial expressions directly reflect psychological states, facial emotion recognition(FER) in particular is one of the most accurate techniques for interpreting human emotions. Traditional text based communication can be changed into emotionally intelligent interaction by integrating FER into chat systems.

The suggested system offers a realtime facial emotion recognition function that can be added as an add-on module to chat programs. In order to compute and normalize geometric and distance based features like eye openness, eyebrow slant, and mouth aspect ratio, 468 facial landmarks are taken from the user's face using a webcam and MediaPipe's Face Mesh. The Cohn Kanade Plus(CK+) dataset is used to train a deep learning model that uses these features to classify seven different emotions which are happiness, surprise, anger, contempt, disgust, sadness, and fear.

When incorporated into a chat program, the system recognizes the user's emotion during a message and instantly displays it as an emoji next to the message. This bridges the gap between human emotions and computer mediated communication by improving emotional awareness, user engagement, and empathy in digital conversations.

II. LITERATURE SURVEY

For more than 20 years, facial emotion recognition(FER) has been a prominent field of study in affective computing and computer vision. It seeks to automatically identify and categorize human emotions from facial expressions, offering vital input for systems that are emotionally intelligent and perceptive. By facilitating automatic feature extraction and hierarchical representation learning from massive facial datasets, recent developments in deep learning have

significantly enhanced FER performance. Convolutional neural networks(CNNs) and attention based models have emerged as the most popular approaches for emotion analysis in recent years, according to Li and Deng's [1] thorough review of deep learning approaches for facial expression recognition.

Deep neural networks for reliable emotion detection in a variety of poses and lighting conditions were investigated in earlier studies, such as those by Mollahosseini et al. [2]. However, the deployment of these image based models in realtime systems is limited because they frequently necessitate substantial computation and sizable annotated datasets. Recent research has investigated geometric and landmark based methods that employ particular facial feature points rather than complete images in order to overcome these limitations. Patel and Mehta [4] showed how realtime emotion detection with a much lower computational overhead could be accomplished by combining machine learning techniques with facial landmarks.

With its extensive collection of labeled facial expressions, the Cohn-Kanade Plus(CK+) dataset, first presented by Lucey et al. [3], has emerged as a benchmark dataset for FER model evaluation. The integration of FER into Human Computer Interaction(HCI) applications was further highlighted by Verma and Gupta [5], who provided examples of how emotion aware systems can improve user engagement and communication efficacy. High fidelity landmark detection is now more accessible for realtime emotion analysis thanks to the creation of lightweight frameworks like Google Research's Face Mesh for MediaPipe [7].

A balanced strategy that guarantees both computational efficiency and accuracy for realtime chat based emotion recognition is offered in this study by utilizing the CK+ dataset [3], MediaPipe landmarks [7], and a deep learning framework [6].

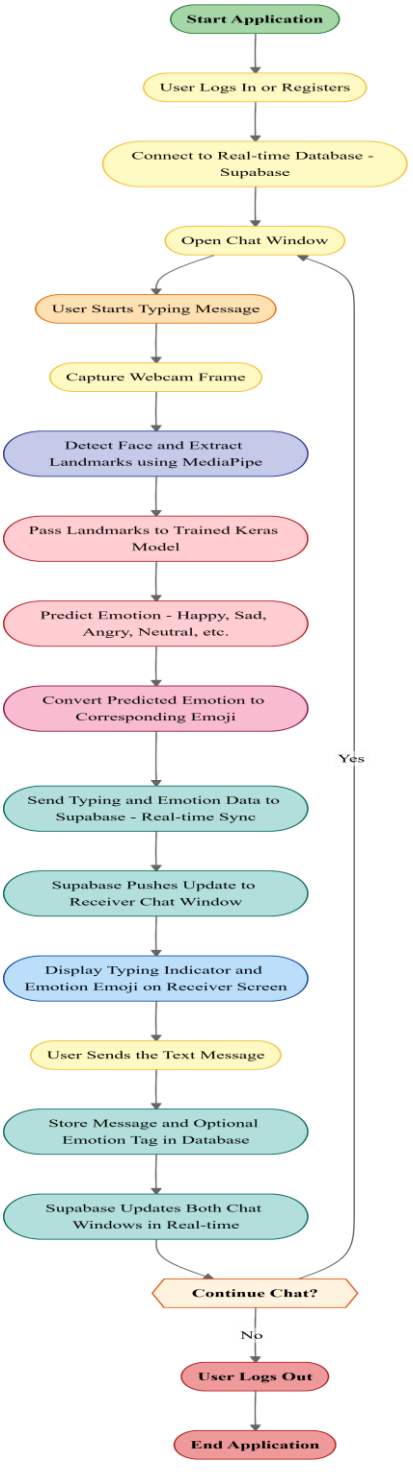
III. METHODOLOGY

In order to identify and show user emotions in real time within a chat application, the suggested facial emotion recognition system combines deep learning and computer vision techniques. Facial landmark extraction, feature computation, emotion classification, and realtime integration into the chat interface are some of the stages that make up the system. Because the entire process is modular, it can be easily implemented on any communication platform.

A. Overview of the System

A realtime chat application built with Supabase as the database for instant data synchronization has a modular extension in the form of the suggested architecture. Because the module functions independently of the main chat logic, the emotion recognition pipeline can run concurrently without compromising latency or message delivery. The architecture is made up of three main parts: (1)Frontend capture module, which connects to the user's webcam and streams frames to the CPU. (2)Emotion detection engine, which uses MediaPipe to extract facial landmarks and feeds the features into a deep learning model for emotion prediction, and (3)Chat interface integration layer, which associates the detected emotion with a corresponding emoji that is dynamically displayed next to the user's message.

The emotion recognition feature can be implemented on desktop, mobile, and web platforms thanks to its modular architecture. In order to maintain low latency and realtime responsiveness, the system's design guarantees asynchronous processing, which means that emotion detection operates in parallel with chat communication. Additionally, the system is computationally efficient due to the use of lightweight geometric features instead of pixel based inputs.

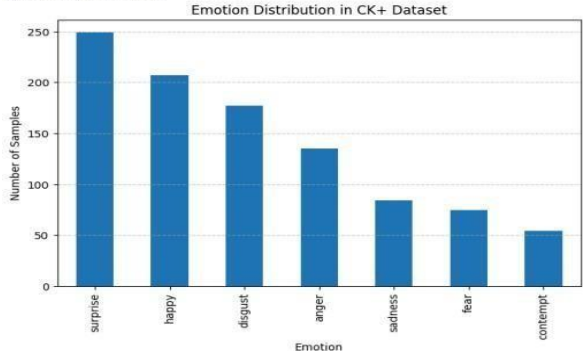


B. Description of the Dataset(CK+ dataset)

The suggested deep learning model was trained and assessed using the Cohn-Kanade Plus(CK+) dataset [3]. CK+, which features high resolution photos depicting a range of emotional states, is a recognized standard in the field of facial expression recognition. Annotated with emotion labels like happiness, surprise, anger, contempt, disgust, sadness, and fear, it consists of 593 video sequences from 123 subjects. Action Units, which are used to record minute facial muscle movements necessary for precise emotion classification, are also included in the dataset.

From a neutral expression to a peak emotional state, each CK+ sequence advances. The pictures were preprocessed by being resized for consistency and converted to grayscale. The MediaPipe's Face Mesh was then used to extract landmarks from each image, yielding 468 unique coordinate points per face. These landmarks formed the basis for classifying emotions by providing the input for the computation of geometric features.

```
Label distribution:
label
surprise    249
happy       207
disgust     177
anger       135
sadness      84
fear         75
contempt     54
Name: count, dtype: int64
```



C. Detecting Facial Landmarks with MediaPipe

In order to extract the geometric features that characterize emotional expressions, facial landmark detection is an important step. The suggested system makes use of MediaPipe's Face Mesh, a realtime framework that offers 468 high fidelity 3D facial landmarks and was created by Google Research [7]. These landmarks allow for in depth geometric analysis by precisely capturing the structure of facial features like the mouth, jawline, eyes, and eyebrows.

The webcam stream is processed frame by frame as the user engages with the chat application. To meet MediaPipe's processing needs, the captured image is first converted from BGR to RGB format. Then, even in the presence of changes in lighting, angle, and facial orientation, the Face Mesh model recognizes facial landmarks. In order to preserve scale invariance across various users and distances from the camera, these coordinates are normalized.

D. Extraction of Features

Following the detection of facial landmarks, geometric and ratio based features that accurately depict emotional states are extracted. The system calculates different aspect ratios and Euclidean distances between important facial points that correspond to important areas such as jaw, eyes, mouth, and eyebrows. To capture changes in facial muscle positions that correspond to various emotions, for example, the mouth aspect ratio(MAR) and eye aspect ratio(EAR) are computed.

Parameters like mouth width, mouth height, eye openness, jaw drop, symmetry of mouth corners, eyebrow slant, eyebrow raise, and nose-mouth distance are among the features that were extracted. To ensure uniformity across various face sizes and orientations, these 14 features are normalized by dividing each by the inter-ocular distance, or the distance between the outer corners of the eyes.

A condensed and discriminative feature set for emotion recognition is offered by this geometric representation. The method drastically lowers dimensionality and gets rid of unnecessary data by avoiding direct pixel-level image processing. The neural network is then fed the resultant feature vector in order to classify it. This design is perfect for real-time applications where responsiveness is essential because it strikes a balance between interpretability and computational efficiency.

E. Architecture of the Deep Learning Model

A fully connected feed forward neural network constructed with TensorFlow and Keras is used to implement the deep learning model. The 14 extracted features are represented by the input layer of the architecture, which is followed by two hidden layers and an output layer. To avoid overfitting, a dropout layer is placed after the first hidden layer, which has 64 neurons with Relu (Rectified Linear Unit) activation. To further improve generalization, a dropout layer comes after the second hidden layer, which has 32 neurons.

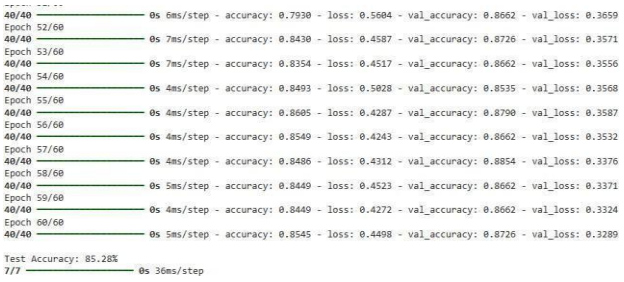
The output layer creates probability distributions for each of the seven emotion classes using a softmax activation function. The sparse categorical cross entropy loss function is used to efficiently handle multiclass classification, and the model is trained using the Adam optimizer, which offers adaptive learning rates for quicker convergence. Class weighting was utilized to address data imbalance across emotion categories during the 60 epochs of training, which

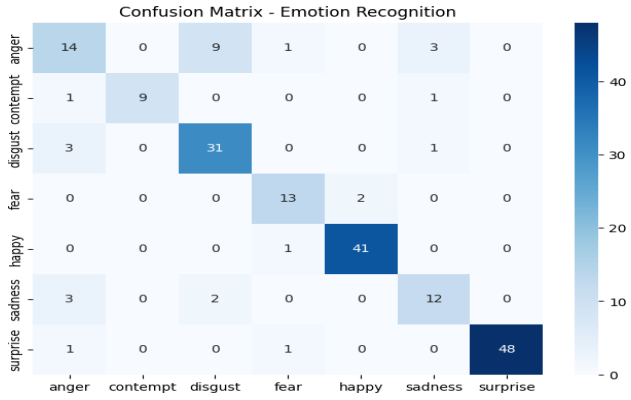
had a batch size of 16. Accuracy and efficiency are balanced in this architecture. With an average test accuracy of 85.28%, the trained model proved its dependability in identifying emotional states using basic geometric features as opposed to computationally costly image representations.

F. Metrics for Training and Assessment

To ensure stratification across all emotion categories, the model was trained using an 80:20 train test split. The training dataset was further split into training and validation sets in an 80:20 ratio to improve generalization. StandardScaler, which standardizes features by taking the mean and scaling them to unit variance, was used to apply data normalization. To reduce bias toward dominant emotion classes, class weights were calculated.

A confusion matrix was used to visualize the classification results, and important metrics like accuracy, precision, recall, and F1 score were used to assess the model's performance. The confusion matrix showed that complex emotions like contempt and disgust were moderately accurately detected, while happiness and surprise were detected with high accuracy. To track training progress and identify possible overfitting, the accuracy and loss curves were plotted.





G. Integrating the model into Chat Application

Integrating the deep learning model that has been trained into the realtime chat system is the last step. The chat application manages user authentication, message storage, and data streaming. It was developed with the help of contemporary web technologies and is powered by Supabase for database synchronization. A frontend extension that uses browser API's to communicate with the user's webcam is used to implement the emotion recognition feature.

The system records live frames and carries out facial landmark detection locally while a user is typing or interacting with the interface during a chat session. The trained model receives the extracted feature vector and makes a realtime emotion prediction. Next, an appropriate emoji is mapped to the predicted emotion and dynamically displayed next to the outgoing message. The emoji enhances emotional awareness by appearing as an expressive visual cue on the receiver's interface.

The emotion detection process operates asynchronously without interfering with message flow thanks to this smooth integration. By enabling users to sense one another's emotional states during conversations, the method fosters emotionally intelligent communication and increases user empathy and engagement in digital communication.

IV. Experimental Results and Analysis

The Cohn-Kanade Plus(CK+) dataset, which consists of pictures depicting seven different emotions - happiness, surprise, anger, contempt, disgust, sadness, and fear were used to train and assess the suggested facial emotion recognition system. After being extracted from MediaPipe's Face Mesh, the geometric features were combined into a feature dataset and split 80:20 between training and testing sets. Z-score normalization was used to standardize each feature in order to guarantee consistent model performance and uniform scaling. During training, class weights were used to reduce imbalances between emotion categories, especially for emotions like fear and contempt that have fewer samples.

With a test accuracy of 85.28%, the deep learning model showed a high degree of generalization across a variety of subjects and emotion types. With training and validation loss steadily declining over 60 epochs, the model converged smoothly during training, suggesting efficient learning and little overfitting. The confusion matrix showed that while visually subtle emotions like disgust and contempt occasionally showed misclassifications due to overlapping facial features, emotions like happiness and surprise were classified with high precision. The classification report also verified that most categories had balanced recall and precision.

Heatmaps and training curves plotted with Matplotlib and Seaborn were used to visually assess performance. Effective class discrimination and model stability were validated by these visualizations. The chat application's

realtime testing also showed low latency, with an average emotion detection and display delay of less than 300 milliseconds, guaranteeing seamless integration during live chat.

Overall, the experimental findings confirm that a compact neural network combined with geometric facial features offers a dependable, portable, and effective realtime facial emotion recognition solution that can be implemented in interactive chat systems.

Classification Report:

	precision	recall	f1-score	support
anger	0.64	0.52	0.57	27
contempt	1.00	0.82	0.90	11
disgust	0.74	0.89	0.81	35
fear	0.81	0.87	0.84	15
happy	0.95	0.98	0.96	42
sadness	0.71	0.71	0.71	17
surprise	1.00	0.96	0.98	50
accuracy			0.85	197
macro avg	0.84	0.82	0.82	197
weighted avg	0.85	0.85	0.85	197

V. Conclusion and Future Scope

In order to improve emotional expressiveness during digital communication, this paper presented a realtime facial emotion recognition system integrated into a chat application. A deep learning model trained on the CK+ dataset was used to process the geometric features that were taken from 468 facial landmarks using MediaPipe's Face Mesh. The model successfully identified seven main emotions with an accuracy of 85.28%. The system bridges the emotional divide in text based communication by adding an empathic layer to standard chat interactions by mapping these emotions to emojis.

Voice and text sentiment analysis can be added to the system in the future to enable multimodal emotion detection. Performance and scalability can be improved across a range of user environments with additional optimization using model pruning, lighting adaptation, and cross dataset training.

VI. Acknowledgment

We would like to sincerely thank our college T. John Institute of Technology's Department of Computer Science and Engineering for their unwavering support, direction, and encouragement during this research. We would especially like to thank Dr. Sudaroli, our project guide, and other faculty members for their insightful comments on system integration and facial emotion recognition.

VII. References

[1] S. Li and W. Deng, *"Deep facial expression recognition: A survey"* IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1195–1215, 2022.

[2] A. Mollahosseini, D. Chan, and M. H. Mahoor, *"Going deeper in facial expression recognition using deep neural networks"* in Proc. IEEE Winter Conf. Applications of Computer Vision (WACV), Lake Placid, NY, USA, 2016, pp. 1–10.

[3] P. Lucey, J. F. Cohn, T. Kanade, J. Saragih, Z. Ambadar, and I. Matthews, *"The extended Cohn-Kanade (CK+) dataset: A complete dataset for action unit and emotion-specified expression"* in Proc. IEEE Conf. Computer Vision and Pattern Recognition Workshops (CVPRW), San Francisco, CA, USA, 2010, pp. 94–101.

[4] M. Patel and S. P. Mehta, *"Real-time emotion detection using facial landmarks and machine learning techniques"* International Journal of Advanced Computer Science and Applications (IJACSA), vol. 12, no. 7, pp. 530–538, 2021

[5] G. Verma and K. Gupta, *"Integrating emotion recognition systems into human-computer interaction applications"* IEEE Access, vol. 9, pp. 145013–145025, 2021.

[6] F. Chollet, *Deep Learning with Python, 2nd ed*. New York, NY, USA: Manning Publications, 2021.

[7] Google Research, *"MediaPipe's Face Mesh: High-fidelity facial landmark detection"* Google AI Blog, 2020.